

When Two Threads Are Better Than One: Modelling HFT Pipeline Latency Under Hawkes Arrivals

Vincent Maciejewski*

September 2026

Abstract

The prevailing view in high-frequency-trading (HFT) system design is that the entire hot path — market-data decode, order-book application, strategy evaluation, order construction — should run on a single-threaded event loop, because each additional thread hop adds wall-clock latency. We show that this reasoning assumes a near-Poisson arrival process, and that it does not hold on CME MDP3 market data, whose packet arrivals are self-exciting (Hawkes-clustered).

Two consequences follow. First, on a clustered feed the end-to-end latency tail is set by cluster-induced queueing, not by the per-hop cost. Second, splitting the servicing chain into concurrent actor stages lowers the latency tail, because the queue that forms behind a burst is drained by several servers in parallel, at the cost of one hop per stage on the median. Above a service-time threshold the tail reduction exceeds the hop cost and the multi-actor pipeline has the lower tail latency; below it the single thread does.

The effect depends on the arrival law. Over the per-message service times an HFT hot path occupies, up to about $20\mu\text{s}$ on this feed, a matched-rate Poisson null has no tail: its p_{99} equals the service time, and splitting adds hop cost only. The clustered feed has a tail over the same band, so the tail reduction in that regime is attributable to clustering. A Poisson process of the same mean rate develops a tail only when utilisation queueing begins; by PASTA the q -quantile of latency equals the service time until the utilisation $\rho = \lambda T$ reaches $1 - q$, which the corpus reproduces to three significant figures at $q = 0.95, 0.99$ and 0.999 . At the corpus-median rate the Poisson p_{99} crossover is at $T \approx 22\mu\text{s}$, about three times the service time at which the clustered feed's tail appears, and past the crossover the clustered feed's tail excess is $11\text{--}45\times$ the Poisson null's. We give an analytic framework for the mechanism, a burst-limit throughput identity and an exact pathwise reduction of the deterministic-service tandem to a single bottleneck server, and test its predictions with simulations driven by a corpus of CME MDP3 packet arrivals. We compare the arrival-process result against an independent measurement on a live production receiver taken seven months after the corpus ends, on instruments the corpus does not contain and with a different arrival clock. The clustering statistics agree, and the production decoder's measured service floor coincides with the service time at which the simulated latency tail appears. Re-running the sweep with service time dependent on the number of messages per datagram leaves the design rule unchanged and isolates a second tail mechanism, in-packet serialisation, which supplies the whole tail below the threshold and accounts for two thirds of the p_{99} excess at the production operating point. Code and cached data are in the KASPAR-HFT repository.

*M2 Tech (16425640 Canada Inc.), Montreal. majeski@gmail.com. KASPAR-HFT is open source at <https://github.com/vincent212/kaspar-hft>.

Contents

1	Introduction	4
1.1	The design choice, and the safe default	4
1.2	Prior recommendations for single-thread hot paths	4
1.3	What this paper does	6
1.4	Contribution and prior-art positioning	7
1.5	Note on the use of AI in this paper	7
1.6	Notation	7
I	Analytical framework	9
2	Analytical framework	9
2.1	Burst-limit throughput theorem	9
2.2	Exact reduction of the tandem to one server, and the scaling bound	11
2.3	What the theory says, in plain terms	14
II	Empirical experiment	17
3	Experiment setup	17
3.1	Data corpus	18
3.2	Packet arrivals are Hawkes-clustered (empirical prerequisite)	19
3.3	Simulator	19
3.4	Pipelining is what makes multi-stage work	20
3.5	Sweep design	21
4	Empirical results	21
4.1	Tail collapse under Hawkes, and its near-absence under Poisson at HFT service times	21
4.2	Median cost is linear in N , exactly as $(N - 1)h$	24
4.3	Design equation	24
4.4	Across windows	26
4.5	Where the tail switches on	27
4.6	Conditioning on intensity and criticality	27
4.7	Sensitivity to h	29
5	Evaluation: do empirics match the analytical predictions?	30
6	Cross-validation: the same arrival process, measured live	32
6.1	Two measurements	32
6.2	What agrees	32
6.3	The service floor lands on the switch-on threshold	33
6.4	Where the two measurements disagree	34
6.5	Per-packet service is not constant	34
6.6	Span-dependent service	35
III	Best practices: implications for HFT systems design	38

7	The reference instance: KASPAR-HFT and its fast_send pipeline	38
8	Implementation on KASPAR-HFT: two dispatch paths, one handler	40
8.1	Receiver transparency	40
8.2	Why measurement, not local reasoning, must decide	41
8.3	Operational discipline	41
8.4	Separation of code construction from latency optimization	41
8.5	Contrast with non-fungible frameworks	42
9	Other architectural considerations to re-evaluate under Hawkes	42
10	Recommendations for HFT system designers	43
10.1	What the paper leaves unchanged	44
10.2	What the paper argues against	44
10.3	Recommended design flow	44
IV	Conclusions	46
11	Conclusion	46
12	Future work	48
13	Reproducibility	48
	Appendices	53
A	The Borel batch-size tail at fixed branching ratio	53
B	Numerical check of Theorem 1	53
C	Numerical check of the unequal-stage claims	54

1 Introduction

1.1 The design choice, and the safe default

The design of the ingress-to-order pipeline in a high-frequency trading system is a choice between two archetypes: the single-threaded event loop that processes every step of every message on one core (SBE decode, book application, strategy evaluation, order construction, all under one call chain, taking a total service time we denote T), and the multi-actor pipeline — a *tandem* of N servers in series — in which those steps are split across N separate actors, each on its own thread with its own mailbox, performing its part of the work and handing the message to the next across one of $N - 1$ explicit hops whose per-hop wall-clock cost we denote h . The low-latency HFT community is predominantly in the single-thread camp, and single-threaded is generally regarded as the safe choice. The argument cites the cost of a mailbox hop (a mailbox enqueue, a cache-line transfer between the sender’s core and the receiver’s core, and the cost of scheduling or spinning the receiver) and holds that $(N - 1)h$ adds to every message’s end-to-end latency, so that each additional stage is a fixed cost that no throughput or tail benefit recovers.

That argument implicitly assumes an arrival process for which single-server throughput $1/T$ is comfortably larger than the peak arrival rate. On CME MDP3 the arrival process is strongly self-exciting: packets arrive in Hawkes-clustered bursts [24–26]. During such bursts the instantaneous arrival rate exceeds the single-thread event loop’s service rate $1/T$, and cluster-induced queueing produces a tail one to two orders of magnitude larger than h . The paper’s central empirical observation, on 276 sessions (3512 thirty-minute windows) of NQ front-month packet arrivals, is that this cluster tail dominates end-to-end latency: the 99th-percentile latency (p_{99}) of a constant-service queue driven by real packet arrivals is 2.2 times the service floor at the corpus-median window for a $16\mu\text{s}$ task and 4.6 times for a $32\mu\text{s}$ task, and 2.5 and 6.1 times at the 95th-percentile window. Under a rate-matched Poisson null of the same arrival count the p_{99} still equals the median at $16\mu\text{s}$ and is only 1.3 times it at 32, against the Hawkes arm’s 4.6; over this band the tail is almost entirely a clustering effect.

The question is then whether splitting the pipeline into $N > 1$ stages, each an independent actor on its own core with per-stage service T/N , reduces the tail, and whether the additional median cost $(N - 1)h$ is small relative to the tail reduction. If both hold, the multi-actor architecture is the better choice for an operator who weights tail latency over median latency.

1.2 Prior recommendations for single-thread hot paths

The single-thread rule has a lineage in the low-latency-systems community, summarised here in reverse chronological order. It has three layers: the HFT-specific codification of the last decade, the general low-latency-systems and event-driven-server culture from which HFT inherited it, and one prominent published counter-argument that the field discarded.

Modern HFT-community codification (2010–present). The founding statement of the current HFT-community formulation is Thompson’s Single Writer Principle [34] — “a single-writer design consistently outperforms a ‘properly’ lock-free or wait-free multi-writer design on most realistic workloads” — motivated by and applied in the LMAX Exchange architecture [35, 36]. Rigtorp’s widely cited low-latency tuning guide [37] formalises the operational rule (“run a single thread in `SCHED_OTHER` per core and use busy waiting/polling”), and Chronicle Software’s public engineering materials [38] adopt the same architecture for market-data connectors (“within a single event loop ... busy-spinning and affinity locks”), pinning Chronicle Queue’s design to a single writer per queue.

The Aeron messaging library [39] inherits the same lineage. The academic side is thinner, but Kwan et al. [40] codify the pattern as the standard low-latency-HFT design pattern, and recent practitioner posts [41] restate it explicitly (“all critical trading logic runs in a single-threaded event loop, eliminating context switches and lock contention entirely”). Inside HFT specifically, the same recommendation has been reinforced by two decades of private prop-trading arms-race practice among the Chicago desks (Getco, Jump, Optiver, DRW; late 2000s), reported second-hand in industry retrospectives but without an open-literature reference.

Older event-driven-server lineage (1995–2010). HFT did not invent this rule; it inherited it from the event-driven-server culture. The single-threaded event loop as an architectural pattern dates at least to Schmidt’s Reactor pattern [42] in the mid-1990s. It was sharpened by Kegel’s C10K essay [43], which named the OS thread-per-connection model as the bottleneck of scalable network servers and argued for event-driven designs. The pattern went mainstream in production with Sysoev’s nginx web server [44], whose single-threaded worker process is explicitly justified as context-switch avoidance, and later with Dahl’s Node.js runtime [45], which pushed the single-threaded event loop into ordinary application code.

Pro-pipelining minority — outside HFT. A legitimate multi-decade pro-pipelining tradition exists in the datacenter and networking-systems literature; it has been the field’s non-single-thread minority throughout the period covered by the previous two paragraphs. The earliest prominent published argument is the Staged Event-Driven Architecture (SEDA) of Welsh et al. [46], which advocated exactly the multi-stage pipeline of independently scheduled event handlers that the present paper’s tandem design revives. Welsh’s 2010 retrospective [47] is often cited as a walk-back to single-thread, but it is actually a hedged pro-pipelining recommendation: it advocates decoupling stages from thread pools, grouping multiple stages within a single thread-pool domain where latency is critical, and putting a separate thread pool and queue in front of any stage with long or nondeterministic runtime — effectively the present paper’s design rule in different vocabulary. Beyond SEDA the pro-pipelining tradition includes the Click Modular Router [48] for staged packet processing; Naiad [49] for low-latency dataflow; Snap [50] for Google’s multi-engine network stack; and the datacenter microsecond-tail-latency scheduler line — Shinjuku [51], Shenango [52], Caladan [53], Perséphone [54] — which argues for centralised multi-core dispatch specifically to control the microsecond-scale tail. Perséphone in particular is targeted at “wide service-time distributions,” the closest existing framing to our arrival-law claim; but the datacenter line frames the problem as heavy-tailed *service*, not clustered *arrivals*. DPDK explicitly supports both run-to-completion and pipeline modes and its documentation recommends pipeline mode “if some set of packets require longer to process” [55]. Inside HFT specifically the pro-pipelining position is rare in the open literature; a Chinese patent [56] describing a pipelined FAST market-data decoder is one commercial instance.

None of the pro-pipelining sources above ties the design recommendation to Hawkes / self-exciting / near-critical arrival dynamics. The closest theoretical hooks are the MMPP-input tandem work of Kim and Chydziński [16] already cited in Section 1.4 and the batch-arrival tandem phase-transition analysis of Vinnikov [57]. The present paper’s mechanism, that the recommendation reverses when arrivals are near-critically self-exciting, does not appear in prior work.

The present author’s earlier work belongs on this list with a qualification.¹ An earlier paper by the present author [32] did not argue for the single-thread principle in its own right. That paper

¹The tail compression reported here had been observed in production before its mechanism was understood: splitting a long task into shorter asynchronously connected stages was known to reduce the tail on a per-strategy basis, and the working assumption at the time was that single-thread implementations were simply faster in the inner loop,

addresses a different objection: the actor model had been considered unsuitable for HFT because of asynchronous-dispatch overhead, and **fast_send** is presented there as the mechanism that removes that overhead for co-located actors, making the actor model usable inside a servicing chain that the prevailing design placed on one thread. Its reference architecture (reader to buffer by asynchronous **send**; decode, book and signal chained by **fast_send** on one thread; signal to order manager by asynchronous **send**) takes the single-thread servicing chain as given and does not compare it against more asynchronous topologies. The present paper revises that design context, not the **fast_send** mechanism: on Hawkes-clustered feeds the servicing chain should be split into asynchronous stages once its service time exceeds the threshold established in Part II. **fast_send** remains the mechanism for chaining actors placed on one thread; what changes is where the thread boundaries inside the chain should be. The design rule is stated in Section 11.

1.3 What this paper does

The paper proceeds in three parts.

- **Theory.** Two closed-form results (Section 2) predict the pipeline behaviour. Proposition 1 shows that under any arrival process an N -stage tandem has an N -fold throughput speedup on isolated bursts. Theorem 1 shows that a deterministic-service tandem reduces exactly to a single server with the largest stage’s service, and that the end-to-end wait is bounded pathwise by $1/N$ of the single-stage wait plus $(N - 1)h$, at every quantile and under every arrival sequence. Corollary 2 shows that under a rate-matched Poisson null the single-stage wait at the q -quantile is already zero whenever the utilisation satisfies $\rho < 1 - q$ — which at this feed’s rate covers p_{99} for every task up to $16\mu\text{s}$ — so over that band there is nothing to compress and splitting costs exactly $(N - 1)h$. The Hawkes–Oakes cluster representation explains why the single-stage wait on a real feed is one to two orders of magnitude above h . A self-exciting process is equivalent to a Poisson stream of immigrant arrivals, each of which generates a cluster of offspring. Cluster size is governed by the branching ratio n , the mean number of direct offspring per arrival: the mean cluster holds $1/(1 - n)$ packets and the largest clusters reach a scale of $(1 - n)^{-2}$ packets (the cluster-size law is the Borel distribution; Appendix A). At the fitted $n \approx 0.8$ the mean is about five and the scale about twenty-five packets, arriving nearly back-to-back. The last member of a burst of K packets waits behind $K - 1$ predecessors on a single server (Proposition 1), so the single-stage wait grows with cluster size; this is the wait the tandem reduces.
- **Simulation.** We build an N -stage tandem Lindley recursion on real CME MDP3 packet-arrival streams at a hop cost $h = 1.7\mu\text{s}$ (the one-way async mailbox cost derived by halving the round-trip figure in Table 4 of Maciejewski [32]) and a service floor swept across $T \in \{2, 4, 8, 16, 32, 64, 128\}\mu\text{s}$, from tasks below the hop cost to tasks far above it. We sweep $N \in \{1, 2, 4, 8\}$ per scenario on each of the 3512 thirty-minute RTH windows of the corpus. Each window is also simulated under a uniform-shuffle Poisson null of the same arrival count as an empirical counterfactual (Section 3.5).
- **Design equation for practitioners.** From the empirical result we extract design rules for a latency-critical servicing chain (Sections 7 and 8): split a stage only when its service time exceeds the feed’s tight-gap floor, about $7.5\mu\text{s}$ on this corpus, since below it there is no queueing tail to compress; choose the partition with the smallest achievable largest stage $s_{\max}$,

not that the recommendation depended on the arrival law. The present paper is the explanation of that observation.

since the tail depends only on $s_{\max}$ and a cut that does not lower it adds a hop and nothing else; place the largest stage first, at ingress; keep sub-tasks below the bottleneck together on one thread via `fast_send`, using an asynchronous `send` only at boundaries that lower $s_{\max}$; and settle each boundary by end-to-end measurement against a representative feed rather than by local reasoning at a single call site.

1.4 Contribution and prior-art positioning

A literature search covering foundational tandem theory (Jackson, Kelly, Burke, Kleinrock, Whitt), tandem theory under bursty arrivals (Foss–Korshunov, Baccelli–Foss–Lelarge, Heindl, Kim–Chydzinski, Dębicki–Mandjes), Hawkes single-queue and adjacent (Daw–Pender, Chen–Blanchet, Koops–Boxma–Mandjes, Sheldon et al., Gao–Zhu, Bo Li, and 2023–2026 follow-ons), and empirical/applied HFT and RPC-pipeline work (ABIDES, ABIDES-MARL, JAX-LOB, Aquilina–Budish–O’Neill, Dean–Barroso, Parsimon, and the datacenter tail-latency literature) found no direct prior art on either of the paper’s two claims. The first is theoretical: that the choice between a single-queue (single-thread) architecture and an N -stage tandem (multi-actor pipeline) is not fixed but *reverses with the arrival law*. At the service times an HFT hot path occupies, single-digit to low-tens of microseconds, Poisson arrivals make the single queue the better choice, because at that utilisation there is no wait to compress and each additional stage adds hop latency only; Hawkes-clustered arrivals at the same mean rate make the tandem better on the tail, because the cluster wait it removes exceeds the hop cost it adds. The reversal is between arrival laws at fixed rate and service time. A Poisson queue also develops a tail at sufficiently long service times, but later and more mildly. We have found no prior work stating that the architectural recommendation reverses between these two arrival laws. The second is empirical: a tandem-queue latency sweep over service floors and stage counts, driven by CME MDP3 packet timestamps, controlled against a rate-matched Poisson null, and calibrated with a hop cost measured on a production HFT actor framework. We have found no prior study that combines a real exchange-calibrated tandem sweep with both that Poisson counterfactual and a measured framework hop cost. The full reference list appears in Section 13.

1.5 Note on the use of AI in this paper

This paper was produced with Anthropic’s Claude (via Claude Code) under the author’s direction for code generation (the analysis pipeline in the KASPAR-HFT repository), exploratory analysis, mathematical drafting, prior-art search, prose editing, and figure preparation. The analytical results of Part I, including the proof of Theorem 1, were drafted with the model and then independently checked by the author. The author independently reviewed all mathematical arguments, experimental methodology, numerical results, and citations, and takes sole responsibility for the contents of the paper.

1.6 Notation

The following symbols are used throughout.

Symbol	Meaning
T	total service time per message on a single-stage pipeline, in μs
N	number of stages in the tandem pipeline (one actor per stage)
T/N	per-stage deterministic service time in the N -stage tandem
h	per-hop wall-clock cost of the async mailbox handoff between adjacent stages
k	size of an arrival burst (number of packets in one cluster)
$\tau_k^{(N)}$	exit time of the last message of a k -burst under N stages
$s_i, s_{\max}$	per-stage services of an unequal tandem, $\sum_i s_i = T$, and their maximum
$w_j(s)$	Lindley waiting time of message j at a single deterministic server with service s
$W_j^{(N)}, W_q^{(N)}$	end-to-end wait of message j (latency minus T , hops included) under N stages, and its q -quantile
p_q	q -th percentile of the end-to-end latency distribution (p_{50}, p_{99} , etc.)
$\Delta(N)$	clustering tail excess $p_{99}(N) - p_{50}(N)$ (§4.3)
$\Delta_{\text{single}}(s)$	single-server clustering tail excess at deterministic service s ; $\Delta(N) = \Delta_{\text{single}}(T/N)$ (§4.3)
γ	fitted exponent in $\Delta(N) \approx \Delta(1)N^{-\gamma}$
κ	operator's tail-versus-median weight in the design equation
N^*	stage count minimising the design objective $(N-1)h + \kappa \Delta(N)$ (§4.3)
μ, α, β	Hawkes background rate, jump size, and decay rate (§3.2)
$n = \alpha/\beta$	Hawkes branching ratio
K	Borel-distributed offspring cluster size (§2.2)
$\bar{\lambda}_{\text{pkt}}$	mean packet-arrival intensity per 30-min window, packets per second
n_{pkt}	branching ratio of the packet-arrival Hawkes fit per window
$n_{\text{pkt}}^{\#}$	per-window packet count
ρ	utilisation, $\rho = \bar{\lambda}_{\text{pkt}} \cdot T$
τ	bin width of the Fano factor, in seconds (distinct from the service time T)
$F(\tau)$	Fano factor at bin width τ : $\text{Var}[C_\tau]/\text{E}[C_\tau]$
H	Hurst exponent, from log-log slope of $F(\tau)$
$\sigma_i, \bar{\sigma}$	span of packet i (SBE messages per datagram) and its window mean (§6.6)
S_i	span-dependent service time of packet i (§6.6)

The per-window packet count and the packet-Hawkes branching ratio would both naturally be written n_{pkt} ; the paper reserves n_{pkt} for the branching ratio and writes $n_{\text{pkt}}^{\#}$ for the count.

Part I

Analytical framework

2 Analytical framework

Throughout, the *tandem* is the paper’s model of the multi-actor pipeline: the total per-message service time T is split into N stages placed in series, each stage an independent deterministic-service server running on its own thread that performs its share of the work and passes the message to the next stage across a mailbox hop of cost h . The single-thread event loop is the $N = 1$ case, in which one server performs all of T ; adding stages shortens each server’s service but inserts a hop between neighbours.

A proposition, a theorem and a corollary underlie the simulation. Proposition 1 is the isolated-burst identity. A message’s end-to-end *latency* is its *service time* T plus its *queueing wait*, the time it spends behind predecessors before service begins. Running the N stages concurrently raises throughput from $1/T$ to N/T , so an N -stage tandem clears a back-to-back burst N times faster: the service time is unchanged at T , and the last message’s queueing wait falls by a factor of N . This is the classical pipeline speedup, stated without proof.

For example, split a service- T server into two equal $T/2$ stages and feed it a burst of k back-to-back messages. All of the queueing now happens at the first stage: its last message waits behind $k - 1$ predecessors that each occupy it for only $T/2$, a wait of $(k - 1)T/2$ against $(k - 1)T$ on the single server. The second stage never queues: stage-one departures arrive spaced $T/2$ apart, equal to its service time, so each message passes through in $T/2 + h$. The wait is not spread across two queues; it remains in one bottleneck stage that is now twice as fast. The halving requires an equal split: cutting $T = 10$ as $7 + 3$ leaves a bottleneck of 7, so the wait falls only to $7/10$. In general the tail scales with the largest stage $s_{\max}$ rather than with the number of stages; Theorem 1 makes this exact. Theorem 1 is the general statement: the tandem’s end-to-end wait equals the wait at a single server with the largest stage’s service plus $(N - 1)h$, exactly and on every sample path, and that single-server wait is at most $1/N$ of the wait at service T . Both are elementary and independent of the arrival law; Theorem 1 rests on a reduction theorem of Friedman [11] and on the monotonicity of the Lindley recursion. Corollary 2 treats the Poisson null: the Poisson wait at quantile q is zero until $\rho = 1 - q$, which at the utilisation of a CME packet feed covers p_{99} for every task up to $16\mu\text{s}$; over that band splitting costs $(N - 1)h$ and removes nothing. The arrival law enters only through the size of the single-stage wait to which the factor $1/N$ is applied, and the Hawkes–Oakes cluster representation explains why that wait is large on a real feed. Part II measures the slack in the bound on real arrivals.

2.1 Burst-limit throughput theorem

Proposition 1 (Burst-limit N -factor speedup). *Consider an isolated cluster of k arrivals whose j -th member arrives no later than $(j - 1)T/N$ after the first, so that no server idles during the burst (an instantaneous batch is the canonical case). Let $\tau_k^{(1)}$ denote the exit time of the last arrival under a single-stage deterministic-service queue with service T , and let $\tau_k^{(N)}$ denote the exit time under an N -stage tandem with per-stage deterministic service T/N and per-stage hop delay h . Then*

$$\tau_k^{(1)} = kT, \quad \tau_k^{(N)} = T + (k - 1)\frac{T}{N} + (N - 1)h, \quad (1)$$

and in particular

$$\lim_{k \rightarrow \infty} \frac{\tau_k^{(1)}}{\tau_k^{(N)}} = N. \quad (2)$$

Remark. *Proposition 1 is the classical pipelined-tandem throughput result [8, 9]; we state it without proof and use it only to fix the baseline against which the arrival-dependent tail reduction of Theorem 1 is measured. What is paper-specific is the numerical calibration of T and h on a production HFT actor framework and the observation that under real CME MDP3 arrivals the ratio $\tau_k^{(1)}/\tau_k^{(N)}$ is what governs the observed tail-latency compression.*

Reading Equation (1). In the paper’s HFT setting we take $T = 10\mu\text{s}$ as a typical HFT task on a single-thread event loop, and k is the size of a Hawkes-cluster burst that hits the pipeline back-to-back. The single-stage formula $\tau_k^{(1)} = kT$ says that in an event-loop design the last message of a k -burst waits behind $k - 1$ predecessors, each of which spends the full T on the one server; a 10-packet cluster on a $10\mu\text{s}$ server therefore has a last-message latency of $100\mu\text{s}$. The tandem formula $\tau_k^{(N)} = T + (k - 1)(T/N) + (N - 1)h$ decomposes the last-message latency into three physical pieces: (i) T , the total service work every message must traverse regardless of N ; (ii) $(k - 1)(T/N)$, the pipelined wait behind $k - 1$ predecessors at throughput N/T rather than $1/T$; and (iii) $(N - 1)h$, the one-time hop-cost tax paid to traverse $N - 1$ mailbox boundaries on the way through. The single message ($k = 1$) pays only (i) and (iii). Under Hawkes arrivals term (ii) is large on a single stage and falls by a factor of N under a tandem, while term (iii) is an additive cost of $(N - 1)h$.

Reading Equation (2). The ratio $\tau_k^{(1)}/\tau_k^{(N)}$ tends to N as the cluster size k grows, meaning that in the burst-saturation limit a tandem of N actors clears the burst N times faster than an event loop. This is the classical pipeline throughput speedup restated for HFT: a 4-actor pipeline drains a large burst in one-quarter of the wall-clock time that a single-thread event loop would take, and a 12-actor pipeline in one-twelfth. The speedup is obtained with the same total service work per message: the pipeline does not reduce the work per event, it overlaps the work across cores so that the queue behind a cluster clears in parallel. The arrival law does not enter Equation (2); what the arrival law controls is *how often* the burst regime is entered, and Theorem 1 shows the same factor governs every message on every sample path.

On what T is, and on the natural stage boundaries. The paper adopts $T = 10\mu\text{s}$ as a ballpark typical HFT single-thread event loop. Any real HFT pipeline of that duration is already a sequential chain of internally atomic sub-tasks (SBE decode, order-book application, strategy touch, order construction on CME MDP3, per Section 8), so the N stages of the tandem are placed at the existing boundaries between those sub-tasks; the paper is not subdividing any single sub-task, and in particular is not subdividing the SBE decode, which is a linear pass over the wire bytes of a single message and cannot be split. Ten microseconds is chosen for exposition; none of the paper’s qualitative results depend on it. The relevant knob for the design equation is the ratio T/h : doubling T at fixed h doubles the cluster-wait terms of Equation (1) and leaves the hop tax unchanged. The floor below which splitting cannot pay (Section 4.3) turns out to be set by the feed rather than by h : it is the arrival stream’s tight-gap floor, and on this corpus it binds well above $2h$. The paper writes “per-stage service T/N ” as a simplifying modelling convention that assumes equal-sized stages; in reality the per-stage services s_i sum to T but are unequal, and the achievable pipeline throughput is $1/\max_i s_i$ rather than N/T . Theorem 1 handles the unequal case exactly: the factor becomes

$s_{\max}/T$ in place of $1/N$, the tail still contracts under Hawkes, and the median still grows linearly at slope h per hop.

The mechanism applies to any deterministic-service tandem regardless of the arrival law; the arrival law determines how often large- k clusters occur. The next subsection makes the per-burst factor a pathwise bound.

2.2 Exact reduction of the tandem to one server, and the scaling bound

Proposition 1 treats one isolated burst. The result that carries the paper's design claim is that the same N -factor holds pathwise for every message under every arrival sequence, at the level of the end-to-end wait. Fix an arrival sequence $a_1 \leq a_2 \leq \dots$ (any point process, clustered or otherwise, with batches allowed as coincident times) and write $\delta_j = a_{j+1} - a_j \geq 0$. For a single FIFO server with deterministic service s fed by this sequence, the waiting time of message j obeys the Lindley recursion [12]

$$w_1(s) = 0, \quad w_{j+1}(s) = (w_j(s) + s - \delta_j)^+. \quad (3)$$

Define the end-to-end wait of message j in the N -stage tandem as $W_j^{(N)} = (\text{exit time}) - a_j - T$, i.e. latency in excess of the total service work; under $N = 1$ this is $W_j^{(1)} = w_j(T)$.

Theorem 1 (Reduction and scaling bound). *Consider an N -stage tandem of FIFO servers with deterministic per-stage services $s_1, \dots, s_N$ summing to T , deterministic hop delay h between consecutive stages, unbounded buffers, and an arbitrary arrival sequence. Let $s_{\max} = \max_i s_i$. Then for every message j and every sample path,*

$$W_j^{(N)} = w_j(s_{\max}) + (N - 1)h, \quad (4)$$

$$w_j(s') \leq \frac{s'}{s} w_j(s) \quad \text{for all } 0 < s' \leq s, \quad (5)$$

and consequently

$$W_j^{(N)} \leq \frac{s_{\max}}{T} W_j^{(1)} + (N - 1)h = \frac{W_j^{(1)}}{N} + (N - 1)h \quad \text{under equal stages } s_i = T/N. \quad (6)$$

Since Equation (6) holds pathwise, it holds for every quantile of the stationary distribution whenever one exists: $W_q^{(N)} \leq W_q^{(1)}/N + (N - 1)h$ for all q . The same inequality holds for the empirical (sample) quantiles computed over any finite trace, directly from the pathwise bound and without invoking stationarity; the sample-quantile form is what Part II measures, and it should be read as a statement about the observed windows rather than about a stationary-distribution quantile or a finite-sample maximum.

Proof. Equation (4), equal stages. Departures from a FIFO server with deterministic service s are spaced at least s apart, because each departure requires s of uninterrupted service after the previous one. A FIFO server with deterministic service s whose arrivals are spaced at least s apart never queues: by induction, message j departs at its arrival time plus s , which is at or before the arrival of message $j + 1$. Stage 1 is a single server with service T/N fed by the original sequence, so message j leaves it at $a_j + w_j(T/N) + T/N$. The arrival sequence at stage 2 is the stage-1 departure sequence shifted by h , spaced at least T/N apart, so stage 2 adds exactly $h + T/N$ and its departures are again spaced at least T/N apart; the same holds at each subsequent stage. Summing, the exit time is $a_j + w_j(T/N) + N \cdot T/N + (N - 1)h$, and subtracting $a_j + T$ gives Equation (4) with $s_{\max} = T/N$.

Equation (4), unequal stages. By the order-invariance theorem of Friedman [11], for a tandem of FIFO stages with constant service times, unlimited queues, and any arrival sequence, each customer's time in system is independent of the order of the stages. Deterministic hop delays preserve this: a hop is a constant shift of one stage's departure sequence into the next stage's arrival sequence, and constant shifts commute with the Lindley recursion, so reordering stages together with their hops leaves every exit time unchanged. Reorder so that the stage with service $s_{\max}$ comes first. Its departures are spaced at least $s_{\max}$ apart, and every subsequent stage has service $s_i \leq s_{\max}$, so by the no-queueing argument above each of them adds exactly $s_i + h$ to every message. The exit time is $a_j + w_j(s_{\max}) + \sum_i s_i + (N - 1)h$, and subtracting $a_j + T$ gives Equation (4).

Equation (5). By induction on j with $w_1(s') = 0 = (s'/s)w_1(s)$. The map $x \mapsto (x + s' - \delta_j)^+$ is nondecreasing, so

$$w_{j+1}(s') = (w_j(s') + s' - \delta_j)^+ \leq \left(\frac{s'}{s}w_j(s) + s' - \delta_j\right)^+ \leq \left(\frac{s'}{s}w_j(s) + s' - \frac{s'}{s}\delta_j\right)^+ = \frac{s'}{s}w_{j+1}(s),$$

where the second inequality uses $\delta_j \geq (s'/s)\delta_j$, which holds because $\delta_j \geq 0$ and $s' \leq s$. Combining Equations (4) and (5) with $s' = s_{\max}$, $s = T$ gives Equation (6). $\square$

Remark (What the theorem says and what it does not). *Equation (4) says all queueing in an equal-stage deterministic tandem happens at stage 1; stages $2, \dots, N$ contribute exactly $T/N + h$ each to every message. Proposition 1 is the isolated-burst special case, where Equation (6) holds with equality. Equation (6) is a bound rather than an identity because the tandem runs its bottleneck stage at utilisation ρ/N while the single stage runs at ρ : the induction step in the proof of Equation (5) gives away $(1 - 1/N)\delta_j$ at every message j that the single server is still serving when message $j + 1$ arrives, so the tandem clears each inter-arrival gap N times more effectively than the scaled bound credits it for. The paper's empirical γ (Section 4.3), defined by $W_q^{(N)} - (N - 1)h \approx N^{-\gamma} W_q^{(1)}$, therefore satisfies $\gamma \geq 1$ in the model, and the excess $\gamma - 1$ measures that slack. Nothing in the theorem depends on the arrival law; what the arrival law determines is the size of $W_q^{(1)}$ that the factor $1/N$ is applied to, which is the subject of the next remark. The hop cost h enters every quantile as exactly $(N - 1)h$, in the median and in the tail alike.*

Remark (Unequal stages: only the largest stage matters). *Real cut points do not give equal stages. Equation (4) states the consequence: the end-to-end wait of any deterministic tandem equals the wait at a single server with service $s_{\max}$, plus h per hop. Three consequences follow.*

1. *The tail factor is $s_{\max}/T$, and the stage count enters only through the hop tax. Splitting a $10 \mu\text{s}$ task as $7 + 3$ compresses the tail excess to at most 70% of single-stage, against 50% for $5 + 5$, and both pay one h .*
2. *The order of the stages is irrelevant to end-to-end latency: $7 + 3$ and $3 + 7$ give identical latencies on every sample path. Order determines only where the queue physically sits (below).*
3. *A cut that does not lower $s_{\max}$ adds a hop and nothing else. Refining $7 + 3$ into $7 + 2 + 1$ leaves every quantile of the wait unchanged and raises end-to-end latency by h . Conversely, sub-tasks can be merged onto one thread (via **fast_send**) without any tail cost as long as the merged service does not exceed $s_{\max}$. The design problem is therefore to choose the partition with the smallest $s_{\max}$ reachable at the natural cut points, and then the fewest hops that realise it.*

Appendix C checks all three numerically.

Remark (Where the burst goes: the bottleneck absorbs the clustering). *The proof of Equation (4) rests on one fact: departures from a deterministic server with service s are spaced at least s apart. Applied to a Hawkes input, that fact says what the bottleneck stage does to the clustering. Upstream of the bottleneck the stream is the raw feed, with intra-cluster inter-arrival times far below $s_{\max}$ and instantaneous intensity far above $1/s_{\max}$. Downstream of the bottleneck, every cluster still leaves as the same group of K messages, so the count-level branching structure is unchanged (the Borel cluster sizes, and the Fano factor at time scales beyond $Ks_{\max}$, are exactly those of the input), but the intra-cluster spacing has been regularised to $s_{\max}$ and the instantaneous intensity is capped at $1/s_{\max}$. The clustering is not removed; it is regularised to a spacing that any consumer at least as fast as the bottleneck can absorb, which is why stages with service $s_i \leq s_{\max}$ never queue. If the bottleneck is stage 1, all of the waiting in the pipeline sits in its mailbox and every downstream mailbox holds at most one message; if the bottleneck is stage $b > 1$, the stages before it queue partially (the raw burst is still bursty relative to their smaller services) and stage b absorbs the remainder, with the same total by Equation (4).*

Four practical consequences follow. First, place the largest stage first, at ingress: end-to-end latency is unchanged by the order, but this confines all queueing to one mailbox, keeps the downstream mailboxes at depth one, and places the burst-absorbing queue where its receiver is continuously busy during a cluster (the regime in which the load-dependent hop cost of Section 4.7 is smallest). Second, any burst-aware logic, such as load shedding, coalescing of superseded book updates, or a cancel-on-burst risk rule, can only observe the raw burst upstream of the bottleneck; downstream it sees a rate- $1/s_{\max}$ stream and cannot tell a cluster from steady flow except by counting. Third, a packet-Hawkes fit run on the bottleneck's departure stream would report a lower intensity peak and a longer effective kernel while recovering the same branching ratio in counts; operators instrumenting a pipeline should fit on the ingress stream. Fourth, deterministic hop delay is what keeps downstream stages queue-free; hop jitter of amplitude ϵ lets a downstream stage queue by at most ϵ per message, so the reduction holds up to that jitter in practice.

Fact (Hawkes–Oakes 1974 cluster representation [2]). *A stationary Hawkes process with background rate $\mu > 0$ and branching ratio $n \in [0, 1)$ is stochastically equivalent to a Poisson-cluster process: a rate- μ Poisson process of immigrant arrivals, each of which independently spawns an offspring cluster whose total size K (including the immigrant) follows the Borel distribution*

$$\Pr(K = k) = \frac{(nk)^{k-1} e^{-nk}}{k!}, \quad k = 1, 2, 3, \dots \quad (7)$$

with mean $E[K] = 1/(1 - n)$ and variance $\text{Var}[K] = n/(1 - n)^3$ [3].

Remark (Where Hawkes enters: the size of the single-stage wait). *Under Poisson arrivals at the utilisation of a CME packet feed ($\rho = \bar{\lambda}_{\text{pkt}}T \ll 1$), the single-stage wait is zero for almost every message, so $W_q^{(1)} = 0$ for every quantile q with $\rho < 1 - q$ and Equation (6) has nothing to compress; that is Corollary 2, and at this corpus's rate it holds at p_{99} for all $T \leq 16 \mu\text{s}$. Under Hawkes arrivals the cluster representation puts K near-coincident messages behind one immigrant, and in the instantaneous-batch picture the last of them waits $(K - 1)T$ on a single stage. Stirling's formula applied to Equation (7) gives, for large k ,*

$$\Pr(K = k) \approx \frac{1}{n\sqrt{2\pi}} k^{-3/2} (ne^{1-n})^k, \quad (8)$$

and $ne^{1-n} < 1$ for every $n < 1$, with $-\log(ne^{1-n}) \approx (1 - n)^2/2$ near criticality. The Borel batch size is therefore light-tailed at every fixed $n < 1$: its survival function follows the critical $k^{-1/2}$ power

law only in the window $k \ll k^* = (1 - n)^{-2}$ and decays geometrically beyond it (Appendix A). The consequences for the paper are, first, that the largest clusters, and hence the single-stage cluster wait they produce, have a characteristic scale of order $Tk^* = T(1 - n)^{-2}$, which at the corpus-median $n \approx 0.8$ (Section 3.2) is $\approx 25T$ and at $n = 0.9$ would be $100T$, one to two orders of magnitude above the service floor and the hop cost; second, that heavy-tail results built on subexponential batch sizes, such as the single-big-batch principle for $M^{[K]}/G/1$ [4] and the monotone-separable tail asymptotics of Baccelli et al. [14], do not apply to this batch distribution at any fixed $n < 1$, so no closed-form power-law exponent for the tail probability should be expected; and third, that the design claim does not depend on any of this, because Theorem 1 applies the factor $1/N$ to whatever $W_q^{(1)}$ the feed produces. Part II measures $W_q^{(1)}$ on real arrivals.

Corollary 2 (Poisson null). *Under homogeneous Poisson arrivals at rate $\bar{\lambda}$ with $\rho = \bar{\lambda}T < 1 - q$, the q -quantile of the single-stage wait is $W_q^{(1)} = 0$, because $\Pr(W^{(1)} > 0) = \rho$ for $M/D/1$ by PASTA. By Theorem 1, $W_q^{(N)} \leq (N - 1)h$, and by Equation (4) $W_q^{(N)} \geq (N - 1)h$, so*

$$W_q^{(N)} = (N - 1)h \quad \text{for all } N \text{ and all } q \leq 1 - \rho. \quad (9)$$

At the utilisations of a CME packet feed (corpus-median 452 packets per second, so $\rho = 0.0036$ at $T = 8\mu\text{s}$) this covers p_{99} at every $T \leq 16\mu\text{s}$ in the paper's sweep: end-to-end latency is $T + (N - 1)h$ at p_{50} and at p_{99} alike, and splitting costs $(N - 1)h$ with nothing to compress. Where ρ/N exceeds $1 - q$, Equation (6) still holds and the tandem reduces the Poisson wait as well; splitting then helps under Poisson for the ordinary utilisation reason rather than for clustering. The sweep contains six such p_{99} cells, all at the long-service end: $(T, N) = (32, 1), (64, 1), (64, 2), (128, 1), (128, 2)$ and $(128, 4)$, spanning $\rho/N = 0.0144$ to 0.0578 . It contains one such p_{95} cell ($T = 128, N = 1, \rho = 0.0578 > 0.05$) and many $p_{99.9}$ cells, and Section 4.1 finds the wait in those cells and in no others.

2.3 What the theory says, in plain terms

This subsection restates Proposition 1, Theorem 1 and Corollary 2 informally. The formal statements above are the reference.

Components of latency. A message's end-to-end latency is the sum of three terms: the work done on it (T in total, however many stages that work is spread over), the hop tax (h for every mailbox boundary it crosses, so $(N - 1)h$ in an N -stage pipeline), and the time it spends sitting in queues waiting for a busy actor. The third term is the wait, W . The work is fixed by the code and the hop cost by the framework; the wait is the only term the architecture changes, and on a bursty feed it dominates the tail.

Proposition 1: one burst. Suppose k messages land at once. On a single thread the last of them waits for the $k - 1$ ahead of it, each costing T : it waits $(k - 1)T$. On an N -stage pipeline the first stage takes only T/N per message, so the last message waits $(k - 1)T/N$ at stage 1 and then flows through the remaining stages without waiting again, because each of them is fed at exactly the rate it can serve. The pipeline clears a burst N times faster for the same total work, because N cores process N different messages at the same instant.

Theorem 1: every message, any arrival sequence. Two facts.

1. In a pipeline where each stage does a fixed amount of work per message, only the slowest stage ever builds a queue. Every other stage receives messages no faster than it can process them, because the slowest stage upstream (or, if the slowest stage is downstream, the stages before it) meters them out. The pipeline therefore behaves as one thread whose per-message work is $s_{\max}$, plus the hop cost. With equal stages, $s_{\max} = T/N$.
2. Take any recording of arrival times, real or synthetic, and replay it through two single-threaded servers: one doing T of work per message and one doing T/N . The second server's queue wait for message number j is never more than $1/N$ of the first server's wait for the same message. The reason is that in the faster server every backlog is N times smaller and every idle gap between arrivals clears at least as much of it.

Combined: for every message in every replay, the pipeline's wait is at most the single-thread wait divided by N , plus $(N - 1)h$. Because this holds message by message, it holds for any summary statistic: median, p_{99} , $p_{99.9}$, or maximum. The bound is met with equality on an isolated instantaneous burst and is strict whenever arrivals are spread out, because the pipeline's first stage is idle more often than the single thread and drains more of its backlog in each idle gap. The measured compression is written $N^{-\gamma}$; the theorem guarantees $\gamma \geq 1$, and Section 4 measures $\gamma = 3.3$ and 2.7 at $T = 64$ and $128 \mu\text{s}$. At shorter service times the tandem reduces the excess to zero and no exponent is defined.

Where the arrival law enters. The arrival law does not enter either theorem. It enters only through the size of the single-thread wait to which the factor $1/N$ is applied. Under Poisson traffic at the load of a CME packet feed, the single thread is idle when almost every message arrives, so for tasks up to $16 \mu\text{s}$ its p_{99} wait is zero; there is nothing to divide by N , and the pipeline is simply $(N - 1)h$ slower (Corollary 2). Under the real feed, packets arrive in self-exciting clusters, the single thread's p_{99} is 2.2 to 41.5 times T in the paper's data for tasks of 16 to $128 \mu\text{s}$, and the pipeline divides the excess by N or better. The Hawkes cluster representation explains the cluster sizes: they follow the Borel distribution, whose large clusters have of order $(1 - n)^{-2}$ packets, about 25 at the corpus's $n \approx 0.8$.

Design rules.

- The tail improvement is set by the slowest stage, $s_{\max}/T$, and by nothing else about the decomposition. Balanced stages are the best case for a given number of hops.
- A cut that does not reduce the slowest stage costs h at every quantile and improves nothing. Sub-tasks can be merged onto one thread until their combined work reaches $s_{\max}$ at no cost to the tail.
- The order of the stages does not change any message's latency. It changes only where the queue physically sits; putting the slowest stage first keeps all queueing in one mailbox.
- Every hop costs h on every message, at the median and in the tail.
- Whether to split reduces to one comparison: the single-thread wait at the quantile of interest, times $(1 - s_{\max}/T)$, against h per hop added. On a Poisson feed at HFT service times the left side is zero; on a clustered feed at the same rate it is tens to thousands of microseconds.

The guarantees above are exact within a specific model. Its assumptions follow.

Fixed work per message. Each stage is assumed to spend the same amount of time on every message. That is what makes only the slowest stage queue: a stage that sometimes takes longer than its upstream neighbour would occasionally fall behind and build its own queue, and then the pipeline’s wait would exceed the single-server wait at $s_{\max}$. The simulator of Part II uses constant per-packet service, so the theorems describe it exactly; Section 6.6 relaxes this for the message count per packet. Real actors also have work that varies with book state, and for them the theorems are a model to be checked by measurement (Section 8).

No drops, no bounds, in-order. Mailboxes are unbounded, messages are never discarded or coalesced, and each stage processes in arrival order. A framework that drops or merges messages under load changes the arithmetic (in the pipeline’s favour, at the cost of lost messages); a priority queue changes which message pays the wait but not the total.

Fixed hop delay. The hop is modelled as a constant h . A jittery hop lets a downstream stage queue by at most the jitter amplitude per message, so the reduction holds up to that amplitude. The paper’s calibration uses the low-load, wake-from-sleep value of h , which Section 4.7 argues is an upper bound inside a burst.

Proposition 1 needs a dense burst. The one-burst identity assumes the k messages arrive faster than the first stage can drain them, i.e. faster than one per T/N . For a burst that trickles in, the ratio in Equation (2) is $kN/(k + N - 1 + N(N - 1)h/T)$, which climbs toward N as the burst grows. Theorem 1 does not need this assumption; it covers every arrival pattern.

What the theory does not predict. It does not say how large the single-thread wait is on a given feed. That number depends on the feed’s clustering and utilisation and, as the remark after the Borel fact explains, has no clean closed form at a fixed branching ratio: the Borel cluster size is light-tailed for every $n < 1$, so the heavy-tail formulas that would give a power-law exponent do not apply. The paper measures the single-thread wait on real CME arrivals instead (Part II). Nor does the theory predict how far below the $1/N$ bound the pipeline lands; that slack is also measured. What the theory guarantees is that the pipeline’s wait is at most $1/N$ of the single-thread wait plus the hop cost, on any arrival sequence.

Part II

Empirical experiment

3 Experiment setup

This section describes the empirical experiment that tests the analytical predictions of Section 2: the data corpus, the confirmation that packet arrivals are Hawkes-clustered (the reason the single-stage wait that Theorem 1 divides by N is large at all), the tandem-Lindley simulator, and the sweep design that produces the numbers reported in Section 4. Figure 1 shows the end-to-end data flow of a single 30-minute window; Algorithm 1 states the full per-window procedure in pseudocode.

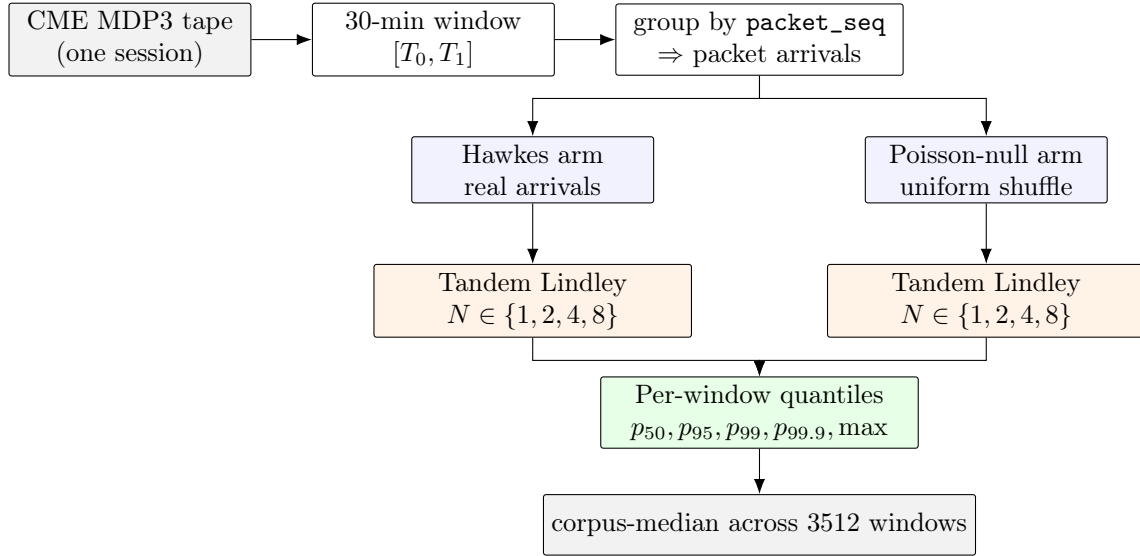


Figure 1. Per-window data flow of the empirical experiment. A single session tape is chopped into 30-minute RTH windows. Each window’s messages are grouped by `packet_seq` into a real packet-arrival stream (Hawkes arm). A rate-matched Poisson-null arm is generated by uniformly shuffling the same arrival count over the same window. Both arms are run through the N -stage tandem Lindley simulator at $N \in \{1, 2, 4, 8\}$. Per-window quantiles are recorded and aggregated by corpus median across all 3512 windows in the 276-session corpus.

Algorithm 1 Per-window procedure of the empirical experiment.

```
1: input: session tape  $S$ , service  $T$ , hop  $h$ , stage counts  $\mathcal{N}$ 
2: for each 30-min RTH window  $W$  in  $S$  with  $[T_0, T_1]$  do
3:    $\{m_i\} \leftarrow$  messages in  $W$ 
4:    $\{(\text{seq}, t)\} \leftarrow \{(\text{packet\_seq}(m_i), \text{sendingTime}(m_i))\}$ 
5:    $\text{arr}_H \leftarrow \text{sort}(\{\min_i t_i : \text{seq}_i = s\}_s)$   $\triangleright$  Hawkes arm (real arrivals)
6:    $n_{\text{pkt}}^\# \leftarrow |\text{arr}_H|$ 
7:    $\text{arr}_P \leftarrow \text{sort}(\text{Uniform}(\min \text{arr}_H, \max \text{arr}_H)^{n_{\text{pkt}}^\#})$   $\triangleright$  Poisson-null arm (seeded per window)
8:   for regime  $\in \{H, P\}$ ,  $N \in \mathcal{N}$  do
9:      $\text{lat} \leftarrow \text{TANDEMLINDLEY}(\text{arr}_{\text{regime}}, N, T, h)$ 
10:    record  $(S, W, \text{regime}, N, \{p_{50}, p_{95}, p_{99}, p_{99.9}, \max\}(\text{lat}))$ 
11:   end for
12: end for
13: return corpus-median of each recorded quantile across all  $(S, W)$  pairs
14: function TANDEMLINDLEY( $\text{arr}, N, T, h$ )
15:    $\text{dep} \leftarrow \text{arr}$ 
16:   for stage  $k = 1, \dots, N$  do
17:     if  $k > 1$  then  $\text{dep} \leftarrow \text{dep} + h$ 
18:     end if
19:      $\text{dep} \leftarrow \text{dep} + \text{LINDLEY}(\text{dep}, T/N)$ 
20:   end for
21:   return  $\text{dep} - \text{arr}$ 
22: end function
```

3.1 Data corpus

The corpus is 281 sessions of CME MDP3 channel 318 (NQ) front-month pcap-derived tapes covering 2025-01 through 2026-02, RTH only, holidays and half-days excluded. Each MBO record carries a `packet_seq` (the UDP packet sequence number) and two nanosecond timestamps: `transactTime`, the CME matching-engine event time carried in the MDP3 message body, and `sendingTime`, the gateway’s per-packet UDP header timestamp. Messages are grouped into packets by `packet_seq`, and a packet’s arrival time is its `sendingTime`.

`transactTime` is not a suitable arrival clock. A single matching-engine transaction is routinely split across several UDP packets that leave the gateway microseconds apart, so all of those packets share one `transactTime`; keying arrivals on it collapses distinct packets onto one instant and produces spurious coincident arrivals. Measured across 178.7 million packets, 3.18% carry a `transactTime` shared with at least one other packet, in groups of up to 66; the true emission spread within such a group has a median of $15.4 \mu\text{s}$ and reaches 51 ms. Because the quantity under study is the spacing of closely packed arrivals, this artefact inflates every clustering and tail statistic. `sendingTime` is emitted once per packet at nanosecond resolution and is the receiving pipeline’s input clock; over the same 178.7 million packets, ordered by `packet_seq`, it produces no ties and no sequence inversions. All results below are keyed on `sendingTime`.

Each session tape is chopped into non-overlapping 30-minute RTH windows; for each window we compute the packet-arrival stream and the per-window quantities listed in Section 1.6. The corpus yields 3512 per-window records from the 276 sessions that contain at least one complete RTH window, roughly 13 windows per session; the remaining five of the 281 tapes are holiday or half-day sessions with none.

3.2 Packet arrivals are Hawkes-clustered (empirical prerequisite)

The paper’s simulator input is the packet-arrival stream, so the first empirical question is whether that stream is clustered or Poisson. Three model-free and one model-based diagnostic on the 276-session, 3512-window NQ front-month corpus reject Poisson. The fraction of interarrival gaps shorter than one-tenth of the mean gap has a corpus median of 73% per 30-minute window, a 5th–95th percentile range of 54–78% and a full range of 38–85%, versus the Poisson benchmark $1 - e^{-0.1} \approx 9.52\%$ that would obtain for any exponential-gap process regardless of rate; not one of the 3512 windows comes within a factor of three of the Poisson value. The Fano factor $F(\tau) = \text{Var}[C_\tau]/\text{E}[C_\tau]$ of packet counts in bins of width τ rises from $F(0.1\text{ s}) = 51.7$ to $F(60\text{ s}) = 1027$ across the corpus median, against $F(\tau) \equiv 1$ for Poisson at every τ ; the log-log fit gives a Hurst exponent $H = 0.74$, against $H = 0.5$ for Poisson. These are within-window figures, corpus-median over the 3512 windows; pooling a whole session instead raises the Fano level by roughly a factor of four without changing the shuffle ratio, for reasons taken up in Section 6.4. Running the identical estimator over the paper’s own Poisson-null arm returns $F(\tau) = 1.00$ at $\tau = 0.1$ through 5 s, 0.94 at 60 s, and $H = 0.50$, which confirms that the estimator is unbiased at every bin width used here and that the Hawkes figures above are not an artefact of binning or of finite window length. Fitting the standard exponential-kernel Hawkes process $\lambda(t) = \mu + \sum_{t_i < t} \alpha e^{-\beta(t-t_i)}$ to each window by MLE (Ogata log-likelihood with the standard recursive intensity, closed-form compensator, Nelder–Mead over (μ, α, β)) gives a branching ratio $n = \alpha/\beta$ with a corpus median of 0.798 and a 5th–95th percentile range of 0.761–0.832 across the 3512 windows (extremes 0.566 and 1.000, the latter a single window pinned at the fit boundary), with every fit converged; the same optimum is recovered by a reparameterised L-BFGS-B fit to four decimals on spot-checked windows. This is the same near-critical regime that Filimonov and Sornette [24] and Hardiman et al. [25] document on ES, at a somewhat lower level than their $n \approx 0.9$; they fit trade arrivals rather than packet arrivals, and a packet aggregates several book events, which is expected to lower the fitted self-excitation. A Fisher–Yates shuffle of the interarrival gap sequence within each window, which preserves the marginal gap distribution exactly and destroys only the ordering, removes most but not all of the excess: the corpus-median Fano at $\tau = 5\text{ s}$ falls from 259.7 to 17.6, and at $\tau = 1\text{ s}$ from 108.4 to 17.5, i.e. 94% and 85% of the excess over the Poisson value of 1. What the shuffle removes is specifically the *scaling*: the shuffled Fano is flat at 16.1–17.6 across every bin width from 0.1 to 60 s, giving $H = 0.50$ against the unshuffled $H = 0.74$. All of the long-range dependence therefore lives in the ordering, which is what distinguishes a self-exciting process from a renewal process. The flat residual of ≈ 17 is a scale-free over-dispersion contributed by the gap marginal itself, which is heavy-tailed; so the corpus is neither a renewal process nor a pure Hawkes process with light-tailed gaps, but a self-exciting process with heavy-tailed gaps. This split bounds what the Hawkes fit describes: the fitted n characterises the ordering component, and the simulator is driven by the recorded arrivals, not by the fitted process. The simulator input is therefore an arrival process far from Poisson, and the matched-rate Poisson null is a strict counterfactual. The Hawkes fit is used only as a description of the observed clustering; the tandem reduction of Section 2 does not depend on the Hawkes assumption, since any arrival process with the same clustering drives the same single-server queue. The model-free diagnostics above, not the fit, establish the empirical premise.

3.3 Simulator

Packet arrivals $t_1 < t_2 < \dots$ enter stage 1. Each stage k is a single-server FIFO queue with deterministic service T/N (own actor, own thread), and departures from stage k are delayed by h

before arriving at stage $k + 1$. Standard Lindley recursion per stage:

$$W_{i+1}^{(k)} = \max\left(0, W_i^{(k)} + \frac{T}{N} - (a_{i+1}^{(k)} - a_i^{(k)})\right), \quad (10)$$

with $a_i^{(k+1)} = a_i^{(k)} + W_i^{(k)} + T/N + h$.

3.4 Pipelining is what makes multi-stage work

The tandem is a *pipelined* design: each stage’s actor runs on its own thread, so at the same wall-clock instant stage 2 can be processing message $k - 1$ while stage 1 is processing message k . The simulator captures this implicitly because Lindley is applied separately per stage and each stage tracks its own “when did the previous message leave” timeline — the two servers’ busy intervals overlap on the wall-clock axis.

Worked example. Three packets arriving at $t = 0, 1, 2 \mu\text{s}$; single-stage total service $T = 8 \mu\text{s}$ (a service floor in the sweep of Table 1); two-stage tandem with per-stage service $T/2 = 4 \mu\text{s}$ and hop $h = 1.7 \mu\text{s}$:

	msg 1	msg 2	msg 3
single-stage arrival	0	1	2
single-stage latency (μs)	8.0	15.0	22.0
tandem arrive stage 1	0.0	1.0	2.0
tandem wait stage 1	0.0	3.0	6.0
tandem leave stage 1	4.0	8.0	12.0
tandem arrive stage 2 ($+h$)	5.7	9.7	13.7
tandem wait stage 2	0.0	0.0	0.0
tandem leave stage 2	9.7	13.7	17.7
tandem latency (μs)	9.7	12.7	15.7

At wall-clock $t \approx 7$, stage 1 is processing message 2 while stage 2 is processing message 1. Message 1 pays the $1.7 \mu\text{s}$ hop cost (latency $8.0 \rightarrow 9.7$); message 3, at the end of the burst, falls from 22.0 to $15.7 \mu\text{s}$. The median rises and the tail falls, at the same message count, with throughput $2/T$ rather than $1/T$. This is the mechanism of Proposition 1.

Pipelining is necessary. The tail-latency improvement in Proposition 1 and Theorem 1 comes entirely from concurrent execution across the N stages: at any wall-clock instant, several different messages are simultaneously in flight at different stages of the pipeline, and it is that overlap that raises system throughput from $1/T$ to $\min_i 1/s_i$ (equal to N/T under equal-sized stages). Without the concurrency, for instance with N synchronous handoffs on a single thread where the sender blocks until each downstream stage finishes, the benefit disappears. In that case each message serially traverses all N stages before the next message can start; total service work per message is still T , so throughput remains $1/T$, and the pipeline adds $(N - 1)h$ of hop cost with no compensating tail win. Under Hawkes bursts a synchronous N -stage chain is strictly worse than a single-stage event loop by exactly $(N - 1)h$ per message. Pipelining, the concurrent execution of different messages at different stages on different threads, is therefore integral to the tandem design; the simulator captures it by applying the Lindley recursion separately per stage, giving each stage its own busy-idle timeline. The framework-implementation Section 8 discusses which KASPAR-HFT dispatch paths actually deliver this concurrency (the asynchronous BQueue path) and which do not (the synchronous `fast_send` path, which despite its low per-call cost does not support pipelining).

Calibration and cost. The paper reports numerical results at $h = 1.7 \mu\text{s}$ — the one-way async mailbox cost obtained by halving the round-trip figure in Table 4 of Maciejewski [32] — swept across $T \in \{2, 4, 8, 16, 32, 64, 128\} \mu\text{s}$ to cover the sub-hop regime through the slow-servicing regime. The simulator is deterministic given the tape and (T, h, N) ; one $O(n)$ pass per stage per window makes its wall-clock cost negligible next to the tape read.

3.5 Sweep design

The design asks whether an N -stage tandem reduces p_{99} latency under real Hawkes packet arrivals and whether the same architecture achieves the same compression under a rate-matched Poisson null. For each 30-minute window we run the tandem Lindley of Section 3.3 at $N \in \{1, 2, 4, 8\}$ under two arrival regimes: the real packet-arrival timestamps from the tape (Hawkes arm) and a uniform-shuffle of the same $n_{\text{pkt}}^\#$ arrivals over the same support (Poisson-null arm). The support is the observed packet span $[\min \text{arr}_H, \max \text{arr}_H]$ rather than the nominal window $[T_0, T_1]$, so the null is rate-matched to the arrivals it replaces rather than to wall-clock; the two coincide on busy windows and the former is the stricter counterfactual on sparse ones. The null is therefore a *conditional homogeneous-Poisson* process: a homogeneous Poisson process at rate $n_{\text{pkt}}^\# / (\max \text{arr}_H - \min \text{arr}_H)$ conditioned on the observed count over the observed support. Per (window, N , regime) we record $p_{50}, p_{95}, p_{99}, p_{99.9}, \max$ of end-to-end latency. Aggregation is corpus-median across all windows; the two headline curves per scenario are $p_{99}(N)$ and $p_{50}(N)$ under Hawkes and Poisson overlaid.

The paper’s empirical calibration is $h = 1.7 \mu\text{s}$ (one-way async cross-thread mailbox cost, per Maciejewski [32]’s Table 4 halved for one-way) swept over $T \in \{2, 4, 8, 16, 32, 64, 128\} \mu\text{s}$ so the scenarios span from the sub-hop regime through the slow-servicing regime. Theorem 1 predicts $p_{99}^H(N) - (N - 1)h - T \leq (p_{99}^H(1) - T)/N$ on every window, i.e. a fitted exponent $\gamma \geq 1$ in $\Delta(N) \approx \Delta(1)N^{-\gamma}$; Corollary 2 predicts $p_{99}^P(N) = p_{50}^P(N) = T + (N - 1)h$ exactly in every cell whose per-stage utilisation satisfies $\rho/N < 0.01$, and a strictly positive Poisson wait in every cell where it does not; the prediction is two-sided, so a Poisson p_{99} sitting on the floor at $\rho/N > 0.01$ would falsify it just as a Poisson p_{99} above the floor at $\rho/N < 0.01$ would. Any of (i) a window on which the Hawkes bound is violated, (ii) a Poisson $p_{99}(N)$ on the wrong side of the $\rho/N = 0.01$ boundary, or (iii) Hawkes $p_{50}(N)$ growing at slope $\gg h$ per stage falsifies the paper’s model of the simulator.

4 Empirical results

4.1 Tail collapse under Hawkes, and its near-absence under Poisson at HFT service times

Table 1 reports corpus-median end-to-end latency quantiles for every (T, N) cell of the sweep at $h = 1.7 \mu\text{s}$, on 276 sessions and 3512 thirty-minute windows (five of the 281 session tapes contain no complete RTH window and are dropped). Figure 2 plots the same numbers. The Poisson-null columns are the uniform-shuffle counterfactual of Section 3.5 with the same packet count per window.

T	N	Hawkes (real arrivals)			Poisson null		
		p_{50}	p_{99}	$p_{99.9}$	p_{50}	p_{99}	$p_{99.9}$
2	1	2.00	2.00	2.00	2.00	2.00	2.00
2	2	3.70	3.70	3.70	3.70	3.70	3.70
2	4	7.10	7.10	7.10	7.10	7.10	7.10
2	8	13.90	13.90	13.90	13.90	13.90	13.90
4	1	4.00	4.00	4.00	4.00	4.00	5.78
4	2	5.70	5.70	5.70	5.70	5.70	5.70
4	4	9.10	9.10	9.10	9.10	9.10	9.10
4	8	15.90	15.90	15.90	15.90	15.90	15.90
8	1	8.00	8.61	10.14	8.00	8.00	13.79
8	2	9.70	9.70	9.70	9.70	9.70	11.48
8	4	13.10	13.10	13.10	13.10	13.10	13.10
8	8	19.90	19.90	19.90	19.90	19.90	19.90
16	1	16.00	34.59	53.97	16.00	16.00	29.84
16	2	17.70	18.31	19.84	17.70	17.70	23.49
16	4	21.10	21.10	21.10	21.10	21.10	22.88
16	8	27.90	27.90	27.90	27.90	27.90	27.90
32	1	32.00	146.14	259.11	32.00	42.02	62.02
32	2	33.70	52.29	71.67	33.70	33.70	47.54
32	4	37.10	37.71	39.24	37.10	37.10	42.89
32	8	43.90	43.90	43.90	43.90	43.90	45.68
64	1	98.88	820.70	1548.68	64.00	106.73	126.72
64	2	65.70	179.84	292.81	65.70	75.72	95.72
64	4	69.10	87.69	107.07	69.10	69.10	82.94
64	8	75.90	76.51	78.04	75.90	75.90	81.69
128	1	364.45	5316.85	12487.72	128.00	237.65	289.07
128	2	164.58	886.40	1614.38	129.70	172.43	192.42
128	4	133.10	247.24	360.21	133.10	143.12	163.12
128	8	139.90	158.49	177.87	139.90	139.90	153.74

Table 1. Corpus-median end-to-end latency quantiles in μs at $h = 1.7 \mu\text{s}$, for each service floor T (μs) and stage count N . 276 sessions, 3512 thirty-minute RTH windows of CME MDP3 NQ front-month packet arrivals. The Poisson null is the uniform shuffle of the same arrival count per window.

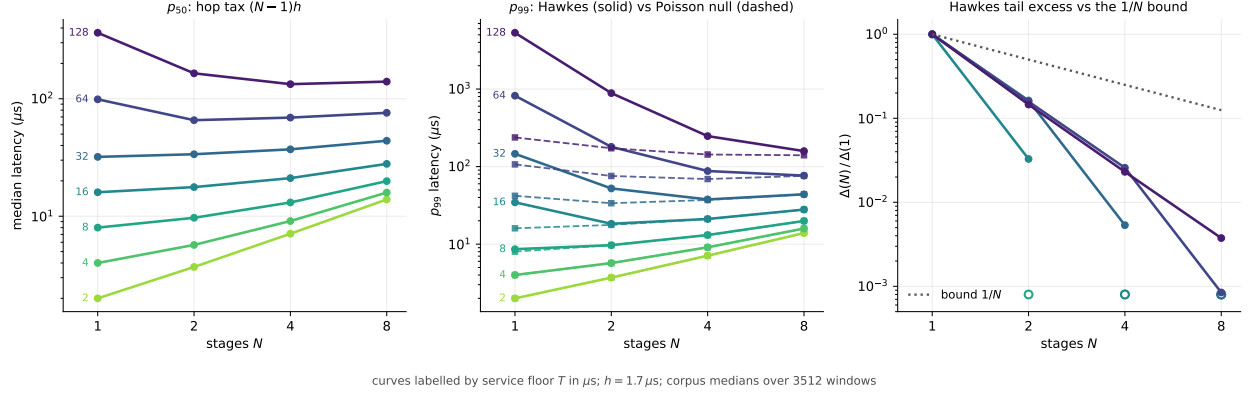


Figure 2. Corpus-median latency versus stage count N for each service floor T (one shade per T , each curve labelled at its $N = 1$ end). Left: median. Centre: p_{99} , Hawkes solid and Poisson null dashed. Right: the Hawkes tail excess ratio $\Delta(N)/\Delta(1)$ against the Theorem 1 bound $1/N$ (dotted); open markers are cells where the tandem drives $\Delta(N)$ to exactly zero, drawn at the panel floor because zero cannot be placed on a log axis. The median rises by exactly h per hop wherever the queue is empty at the median, which is every cell except the three longest-service Hawkes cells. The Hawkes p_{99} falls with N only at $T \geq 16 \mu\text{s}$; at $T \leq 8 \mu\text{s}$ there is no tail to compress and it rises with the hop count, tracking the median. The Poisson p_{99} likewise tracks the median up to $T = 16 \mu\text{s}$ and separates from it at $T \geq 32 \mu\text{s}$, where utilisation queueing begins.

Under the Poisson null the queue is empty when almost every message arrives, so every reported quantile equals the service floor, $p_{50} = p_{99} = T + (N - 1)h$, in every cell with $T \leq 16 \mu\text{s}$. The first exception is $T = 32 \mu\text{s}$, $N = 1$, where Poisson p_{99} is $42.0 \mu\text{s}$ against a floor of 32: at the corpus-median packet rate of 452 packets per second the utilisation there is $\rho = 452 \times 32 \mu\text{s} = 0.0145$, above the 0.01 at which Corollary 2 says the Poisson wait must first appear at p_{99} , and it does; at $N = 2$ the per-stage utilisation halves to 0.007 and the wait vanishes again. Beyond that the null arm is a perfectly ordinary queue: at $T = 64$ and $128 \mu\text{s}$ the single-stage Poisson p_{99} reaches 106.7 and $237.7 \mu\text{s}$, and splitting reduces it, to 75.7 and $172.4 \mu\text{s}$ at $N = 2$. Under Poisson, splitting is therefore not useless in general; it is useless in the regime an HFT hot path occupies. The $p_{99.9}$ column shows the same pattern one decade lower: the Poisson wait is visible at $N = 1$ for every $T \geq 4 \mu\text{s}$ (single-stage $\rho > 10^{-3}$) and at $N = 2$ for $T \geq 8 \mu\text{s}$, and absent wherever the per-stage ρ/N falls below 10^{-3} . Every one of these switch-on points is the $\rho = 1 - q$ boundary of Corollary 2, which the corpus reproduces to three significant figures at $q = 0.95$, 0.99 and 0.999 (Section 4.6). The null arm is a queue with a small, predictable wait, and over $T \leq 16 \mu\text{s}$ splitting under Poisson costs $(N - 1)h$ at every reported quantile.

Under Hawkes a tail appears only above a threshold. At $T = 2$ and $4 \mu\text{s}$ there is no tail: single-stage p_{99} and $p_{99.9}$ both equal the service floor, the clustered feed is indistinguishable from the Poisson null, and splitting costs $(N - 1)h$ and removes nothing. The tail appears between $T = 4$ and $8 \mu\text{s}$ and then grows rapidly: single-stage p_{99} is 1.00, 1.00, 1.08, 2.16, 4.57, 12.8 and 41.5 times the service floor at $T = 2, 4, 8, 16, 32, 64, 128 \mu\text{s}$, and single-stage $p_{99.9}$ reaches $97.6 T$ at $T = 128$. Splitting into two stages cuts the p_{99} from 34.59 to $18.31 \mu\text{s}$ at $T = 16$, from 146.1 to $52.3 \mu\text{s}$ at $T = 32$, from 820.7 to $179.8 \mu\text{s}$ at $T = 64$, and from 5316.9 to $886.4 \mu\text{s}$ at $T = 128$, in each case for a median cost of $1.7 \mu\text{s}$ per hop; the two-stage p_{99} is below the single-stage p_{99} on 100% of the 3512 windows at every $T \geq 16 \mu\text{s}$. The stage count that minimises p_{99} rises with T : $N = 1$ at $T \leq 8$, $N = 2$ at $T = 16$, $N = 4$ at $T = 32$, $N = 8$ at $T = 64$ and 128 . The $T = 2$ and $4 \mu\text{s}$ rows, both below the floor, are included to demonstrate that boundary: splitting there is dominated under both arrival laws. Section 4.5 resolves the 4–8 μs onset to $1 \mu\text{s}$ and Section 4.6 explains what sets its location.

4.2 Median cost is linear in N , exactly as $(N - 1)h$

The p_{50} columns of Table 1 match the single-message formula of Proposition 1

$$p_{50}(N) = T + (N - 1)h \quad \text{with } h = 1.7 \mu\text{s},$$

in all 28 cells of the Poisson arm and in 25 of the 28 Hawkes cells — for instance 8.00, 9.70, 13.10, 19.90 at $T = 8$. Under Poisson the identity holds because the queue is empty; under Hawkes it holds because the median message arrives in a quiet inter-cluster stretch where the queue is also empty. The hop tax is therefore exactly h per boundary on the median, with no hidden component, as Theorem 1 says it must be at every quantile.

The three exceptions are at the long-service end of the sweep: $(T, N) = (64, 1)$, $(128, 1)$ and $(128, 2)$, where the Hawkes p_{50} is 98.88, 364.45 and 164.58 μs against floors of 64.00, 128.00 and 129.70. There the service time is long enough that the median message no longer arrives in a quiet stretch: clusters merge into a persistent backlog and the typical message waits. This is the regime boundary appearing in the median rather than in the tail, and it lies outside the range the paper’s recommendations address. Splitting restores the identity once the per-stage service is below the threshold; at $T = 128$ the median is on the floor by $N = 4$.

4.3 Design equation

From the corpus:

$$p_{99}^{\text{Hawkes}}(N) = T + (N - 1)h + \Delta(N), \quad (11)$$

$$p_{50}^{\text{Hawkes}}(N) = T + (N - 1)h, \quad (12)$$

where $\Delta(N) \equiv p_{99}^{\text{Hawkes}}(N) - p_{50}^{\text{Hawkes}}(N)$ is the “clustering tail excess”. Table 2 lists it, together with the ratio $\Delta(N)/\Delta(1)$ that Theorem 1 bounds by $1/N$.

T	$\Delta(N)$ (μs)				$\Delta(N)/\Delta(1)$			γ
	$N=1$	$N=2$	$N=4$	$N=8$	$N=2$	$N=4$	$N=8$	
2	0.00	0.00	0.00	0.00	—	—	—	—
4	0.00	0.00	0.00	0.00	—	—	—	—
8	0.61	0.00	0.00	0.00	0.000	0.000	0.000	—
16	18.59	0.61	0.00	0.00	0.033	0.000	0.000	—
32	114.14	18.59	0.61	0.00	0.163	0.005	0.000	—
64	721.82	114.14	18.59	0.61	0.158	0.026	0.001	3.32
128	4952.41	721.82	114.14	18.59	0.146	0.023	0.004	2.68
bound					0.500	0.250	0.125	≥ 1

Table 2. Corpus-median clustering tail excess $\Delta(N) = p_{99}(N) - p_{50}(N)$ under Hawkes arrivals at $h = 1.7 \mu\text{s}$, its ratio to the single-stage value, and the least-squares exponent γ in $\Delta(N) \approx \Delta(1)N^{-\gamma}$. The last row is the Theorem 1 bound.

Two features of Table 2 follow from the theorems. First, Δ depends on (T, N) only through the per-stage service T/N : every diagonal of constant T/N carries one number — 0.00 at $T/N \leq 4 \mu\text{s}$, 0.61 at $T/N = 8$, 18.59 at 16, 114.14 at 32, 721.82 at 64 — to the reported precision, across up to four different (T, N) pairs, without fitting. This is the reduction identity Equation (4) on real

arrivals: the tandem’s tail is the single-server tail at $s_{\max}$, and the table is a table of $\Delta_{\text{single}}(s)$ at $s \in \{0.25, 0.5, \dots, 64, 128\} \mu\text{s}$, read off the diagonals.

Second, every ratio is at or below the bound $1/N$, and the pathwise check of Section 5 finds no message on any window above it. The margin is large, because of the threshold. Because $\Delta_{\text{single}}(s)$ is identically zero for $s \lesssim 7.5 \mu\text{s}$ (Section 4.5), a split that pushes the per-stage service below that floor does not merely divide the tail excess, it removes it. At $T = 32$, Δ falls $114.14 \rightarrow 18.59 \rightarrow 0.61 \rightarrow 0.00$ as N goes $1 \rightarrow 2 \rightarrow 4 \rightarrow 8$: a ratio of 0.005 at $N = 4$ against a bound of 0.25. The fitted exponent γ in $\Delta(N) \approx \Delta(1)N^{-\gamma}$ is correspondingly steep, 3.32 at $T = 64$ and 2.68 at $T = 128$, well above the $\gamma = 1$ that the bound alone would permit; at $T \leq 32$ the tandem drives $\Delta(8)$ to exactly zero and no power law is fitted, since the least-squares slope would be reading the numerical floor rather than the data. The decay of the excess is therefore not a $1/N$ division on this corpus; it terminates at zero once the per-stage service is below the floor. The design consequence is that the objective is not a large N but an $s_{\max}$ below the feed’s tight-gap floor. The utilisation slack described after Theorem 1 contributes as well, and Appendix B reproduces it on synthetic Borel clusters.

The operator’s stage-count optimum is

$$N^* = \arg \min_N \left[(N-1)h + \kappa \Delta(N) \right]$$

for the operator’s tail weight κ (the number of microseconds of median the operator will spend to remove one microsecond of p_{99} excess). Over the simulated grid $N \in \{1, 2, 4, 8\}$ at $h = 1.7 \mu\text{s}$ the breakpoints are:

T (μs)	$N^* = 1$	$N^* = 2$	$N^* = 4$	$N^* = 8$
2	always	—	—	—
4	always	—	—	—
8	$\kappa < 2.8$	$\kappa \geq 2.8$	—	—
16	$\kappa < 0.095$	$0.095 \leq \kappa < 5.6$	$\kappa \geq 5.6$	—
32	$\kappa < 0.018$	$0.018 \leq \kappa < 0.19$	$0.19 \leq \kappa < 11$	$\kappa \geq 11$
64	$\kappa < 0.0028$	$0.0028 \leq \kappa < 0.036$	$0.036 \leq \kappa < 0.38$	$\kappa \geq 0.38$
128	$\kappa < 0.0004$	$0.0004 \leq \kappa < 0.0056$	$0.0056 \leq \kappa < 0.071$	$\kappa \geq 0.071$

A dash means that stage count is never optimal at any κ : at $T \leq 8 \mu\text{s}$ the tail excess is already zero by $N = 2$, so further splitting can only add hop cost. An operator who values tail and median equally ($\kappa = 1$) lands on $N^* = 1$ at $T = 2, 4$ and 8, 2 at $T = 16$, 4 at $T = 32$, and 8 at $T = 64$ and $128 \mu\text{s}$, which is also the ordering of the raw p_{99} minima in Table 1. Over the service times an HFT hot path occupies, an operator must weight the tail several times more than the median before splitting pays; above the threshold the required weight is small.

At real cut points the stages are unequal, and by Remark 2.2 the tail excess depends on the partition P of the servicing chain only through its largest stage, $\Delta(P) = \Delta_{\text{single}}(s_{\max}(P))$, i.e. the single-stage tail excess measured at service $s_{\max}$. The optimum over partitions is then

$$P^* = \arg \min_P \left[\text{hops}(P)h + \kappa \Delta_{\text{single}}(s_{\max}(P)) \right],$$

which Table 2 lets the operator evaluate directly, since its diagonals are $\Delta_{\text{single}}(s)$ on a power-of-two grid of s . Two rules follow. A cut is worth its hop only if it lowers $s_{\max}$ by enough that $\kappa [\Delta_{\text{single}}(s_{\max}) - \Delta_{\text{single}}(s'_{\max})] > h$; and among partitions with the same $s_{\max}$ the one with the fewest hops is preferred, so sub-tasks downstream of the bottleneck should be merged onto one thread until their sum reaches $s_{\max}$.

Design principle (only $s_{\max}$ matters). For a servicing chain cut at fixed boundaries, the tail-latency question is not “how many threads?” but “what is the smallest achievable largest stage $s_{\max}$?” By Theorem 1 the clustering tail excess of any partition is $\Delta_{\text{single}}(s_{\max})$, independent of the stage count; a cut that does not lower $s_{\max}$ compresses no tail and still costs one hop h . Choose the partition with the smallest reachable $s_{\max}$, then the fewest hops that realise it.

Floor on T : short tasks cannot be usefully split. Equations (11)–(12) imply a lower bound on the tasks the recommendation applies to. Splitting a task of total service T into N stages spends $(N - 1)h$ at every quantile to compress a tail excess $\Delta(1)$ that is itself a strongly increasing function of T (0 at $T \leq 4 \mu\text{s}$, rising to $39T$ at $T = 128$, Table 2). When T is at or below the feed’s tight-gap floor there is no tail excess to remove at all, and the design equation prescribes $N^* = 1$; a hop there adds cost only. The sweep confirms this: at $T = 2$ and $4 \mu\text{s}$, $\Delta(1) = 0$, so a split removes nothing and p_{99} rises by the full hop, from 2.00 to 3.70 and from 4.00 to 5.70. Above the floor: at $T = 16 \mu\text{s}$, $\Delta(1) = 18.59 \mu\text{s}$, of which the split removes 17.98 for the same 1.7, and p_{99} falls from 34.59 to 18.31 on every window.

Two separate necessary conditions apply. The first is a property of the *framework*: a split cannot pay unless the tail excess it removes exceeds the hop cost it adds, which on this framework’s shipping BQueue path ($h \approx 1.7 \mu\text{s}$) puts a floor at roughly $T \gtrsim 2h \approx 3.4 \mu\text{s}$. The second is a property of the *feed*: there is no tail excess to remove at all until T exceeds the arrival stream’s tight-gap floor, about $7.5 \mu\text{s}$ on this corpus (Section 4.6). The binding constraint is whichever is larger, and on CME MDP3 channel 318 with a BQueue mailbox it is the feed’s, by roughly a factor of two; this is why $T = 4 \mu\text{s}$, although above $2h$, has $\Delta(1) = 0$ and should not be split. A framework with a much more expensive mailbox would find its own floor binding instead. Throughout the rest of the paper, “the floor” means the larger of the two. Sub-microsecond tasks, for instance a single SBE integer decode or a small handler that fires from a strategy fast path, belong on `fast_send` rather than on a cross-thread mailbox, regardless of feed statistics.

4.4 Across windows

The corpus medians understate the tail on the busiest windows. Across the 3512 windows the single-stage p_{99}/T has a median of 1.0, 1.0, 1.1, 2.2, 4.6, 12.8, 41.5 and a 95th percentile of 1.0, 1.0, 1.1, 2.5, 6.1, 19.9, 81.7 at $T = 2, 4, 8, 16, 32, 64, 128 \mu\text{s}$, so one window in twenty at $T = 32$ has a single-stage p_{99} above $195 \mu\text{s}$ and at $T = 128$ above 10.5 ms. The tandem’s win is not confined to the median window: $p_{99}(2) < p_{99}(1)$ and $p_{99}(4) < p_{99}(1)$ both hold on 100% of windows at every $T \geq 16$, and on no window at all at $T \leq 8$, where there is no tail to compress and the split pays only hop cost. The crossover is sharp and sits just below $T = 10 \mu\text{s}$: on the refined sweep of Section 4.5, $p_{99}(2) < p_{99}(1)$ on 0% of windows at $T = 8 \mu\text{s}$, 67.0% at $T = 9 \mu\text{s}$ and 99.7% at $T = 10 \mu\text{s}$. The per-window fitted branching ratio has median 0.798 and a 5th–95th percentile range of 0.761–0.832 (Section 3.2); how the tail excess depends on it, and on the window’s packet rate, is examined in Section 4.6. The intervening p_{95} quantile is recorded per window in the sweep (Section 3.5) and omitted from the tables above only for brevity; it lies between the reported p_{50} and p_{99} in every cell by construction, and under the Poisson null it equals the service floor wherever p_{99} does, since $\rho/N < 1 - 0.99$ implies $\rho/N < 1 - 0.95$. The 3512 windows are treated as evaluation units for the descriptive statistics reported here; they are nested within 276 sessions and are not independent, so any statistical uncertainty on the aggregates would be clustered at the session level rather than treating 3512 as an independent sample size.

4.5 Where the tail switches on

The coarse sweep brackets the onset of the clustering tail between $T = 8$ and $T = 16 \mu\text{s}$, too wide for a design rule. We reran the single- and two-stage arms on the same 3512 windows at $T \in \{5, 6, 7, 8, 9, 10, 12, 14, 16\} \mu\text{s}$:

T (μs)	p_{99} Hawkes	excess over T	windows with a tail	$p_{99}(2) < p_{99}(1)$
5	5.00	0.00	0.0%	0.0%
6	6.00	0.00	0.0%	0.0%
7	7.00	0.00	4.5%	0.0%
8	8.61	0.61	98.6%	0.0%
9	10.85	1.85	100.0%	67.0%
10	13.52	3.52	100.0%	99.7%
12	19.83	7.83	100.0%	100.0%
14	26.68	12.68	100.0%	100.0%
16	34.59	18.59	100.0%	100.0%

Two distinct thresholds are visible. The first is the *tail* threshold: between $T = 7$ and $T = 8 \mu\text{s}$ the fraction of windows with any p_{99} excess jumps from 4.5% to 98.6%. The second is the *splitting* threshold, about $1.5 \mu\text{s}$ later, between $T = 8$ and $T = 10 \mu\text{s}$: a tail must exist *and* exceed the hop cost before a split repays it. At $T = 8 \mu\text{s}$ nearly every window has a tail, but its median excess is $0.61 \mu\text{s}$, well under $h = 1.7 \mu\text{s}$, and splitting loses on every window. By $T = 10 \mu\text{s}$ the excess is $3.52 \mu\text{s}$, above h , and splitting wins on 99.7%.

The first threshold is set by the feed’s tight-gap structure rather than by its mean rate; the next section establishes this.

4.6 Conditioning on intensity and criticality

The results above are conditioned on the service time T alone. A Hawkes tail depends on two further quantities that the corpus median averages over: the window’s mean packet rate $\bar{\lambda}$, and its branching ratio $n = \alpha/\beta$. The expectation is that the tail, and therefore the case for splitting, increases with both, so that a rule stated in T alone would average over market states. This section tests that expectation; it is not borne out over the service times an HFT hot path occupies.

We sort the 3512 windows into quintiles, first by $\bar{\lambda}$ and then by the fitted n , and report the corpus median of the normalised single-stage tail p_{99}/T within each quintile. Normalising by T removes the service floor, so a value of 1.0 means no tail at all and the quintile-to-quintile ratio isolates the conditioning effect.

quintile	median	p_{99}/T , corpus median				
		$T=8$	$T=16$	$T=32$	$T=64$	$T=128$
<i>by packet rate $\bar{\lambda}$ (pkt/s)</i>						
Q1	202	1.08	2.29	4.74	12.25	31.76
Q3	451	1.07	2.12	4.48	12.68	42.81
Q5	942	1.08	2.11	4.51	13.92	55.67
Q5/Q1		1.00	0.92	0.95	1.14	1.75
<i>by branching ratio n</i>						
Q1	0.769	1.08	2.18	4.51	11.61	30.73
Q3	0.798	1.08	2.16	4.61	13.17	42.83
Q5	0.825	1.07	2.15	4.61	14.65	58.73
Q5/Q1		1.00	0.98	1.02	1.26	1.91

The result divides at $T \approx 32\text{--}64\mu\text{s}$. For $T \leq 32\mu\text{s}$ the normalised tail is flat in both variables: a $4.66\times$ change in packet rate moves it by 0–8%, and in the wrong direction at $T = 16$ and $T = 32$, where the *busiest* quintile has the slightly *smaller* normalised tail. A $0.769 \rightarrow 0.825$ change in branching ratio moves it by at most 2%. For $T \geq 64\mu\text{s}$ both variables have an effect, in the expected direction: by $T = 128\mu\text{s}$ the busiest quintile’s tail is $1.75\times$ the quietest and the most critical quintile’s is $1.91\times$ the least. This is the boundary between the clustering regime and the utilisation regime found elsewhere in the paper: for $T \geq 64\mu\text{s}$ the Poisson null has a tail too (Section 4.1), utilisation $\rho = \bar{\lambda}T$ is the governing quantity, and ρ is proportional to $\bar{\lambda}$. Intensity matters where utilisation queueing applies and not below it.

Why intensity has no effect below the crossover. The null result has a mechanical explanation in the arrival stream. Taking the same $\bar{\lambda}$ quintiles and reporting the corpus median of the per-window interarrival-gap quantiles:

quintile	$\bar{\lambda}$ (pkt/s)	p_1 gap	p_5 gap	p_{25} gap	p_{50} gap	mean gap
Q1	202	7.45	8.85	21.30	87.10	4944
Q2	320	7.52	8.75	21.87	84.52	3128
Q3	451	7.53	8.70	22.70	82.16	2215
Q4	620	7.51	8.66	23.36	81.98	1612
Q5	942	7.43	8.58	22.55	76.52	1062
Q5/Q1	4.66×	0.997×	0.969×	1.059×	0.878×	0.215×

All gaps in microseconds. The mean gap falls by exactly the reciprocal of the rate increase, $0.215 \approx 1/4.66$, which is definitional. Every other column is approximately invariant, and the tightest is the most invariant: the p_1 gap moves from 7.45 to $7.43\mu\text{s}$, a ratio of 0.997, while the rate more than quadruples.

Raising the intensity of a self-exciting process raises the rate at which clusters are generated; it does not change the spacing of events inside a cluster. The intra-cluster spacing is set by the exchange’s packet-emission granularity, a property of the CME gateway rather than of the market state, and on this corpus that floor is about $7.5\mu\text{s}$. A busy window contains more bursts, not tighter bursts.

The queueing consequence: a deterministic server of service time T begins to build a queue when it is handed two arrivals separated by less than T , so the service time at which a tail appears is set by the *tight* end of the gap distribution, not by its mean. The tight end is invariant to $\bar{\lambda}$; therefore the threshold is invariant to $\bar{\lambda}$. The two numbers agree: the measured p_1 gap of $7.43\text{--}7.53\mu\text{s}$ across every intensity quintile brackets the measured tail onset of Section 4.5, between $T = 7$ and $T = 8\mu\text{s}$, where the fraction of windows carrying any tail jumps from 4.5% to 98.6%.

Robustness to window length. A 30-minute window averages $\bar{\lambda}$ over a long interval, and the quintile split above may be comparing time-averaged rates that have smoothed away the relevant intensity variation. We therefore reran the entire pipeline — packet extraction, per-window Hawkes MLE, Poisson null and the full (T, N) grid — on *five*-minute windows, giving 21,060 windows in place of 3512. Shorter windows do resolve more intensity variation, as expected: the corpus $\bar{\lambda}$ spread widens from $6.8\times$ to $7.3\times$ between the 5th and 95th percentiles, from $26.0\times$ to $30.6\times$ between the 1st and 99th, and the observed maximum rises from 2152 to 2692 packets per second; the 5th–95th range of the fitted branching ratio widens from 0.0717 to 0.0786. The conditioning result is unchanged to two decimal places across the sweep:

	$\bar{\lambda}$ Q5/Q1	$T=8$	$T=16$	$T=32$	$T=64$	$T=128$
30-min windows ($n = 3512$)	$4.66\times$	1.00	0.92	0.95	1.14	1.75
5-min windows ($n = 21,060$)	$4.98\times$	1.00	0.93	0.96	1.15	1.74

Entries are the Q5/Q1 ratio of the normalised single-stage tail p_{99}/T . Resolving intensity six times more finely, over a wider realised range, moves no entry by more than 0.01. The result is a property of the arrival process, not of the window length.

Consequence for the design rule. The rule (split a stage whose service time exceeds the feed’s tight-gap floor; do not split below it) can therefore be stated as a constant of the feed rather than as a function of market state. It requires no runtime intensity estimate and does not change between a quiet and a volatile session. This holds because of the invariance measured above, and should be rechecked on any feed whose packet-emission behaviour differs from CME MDP3’s, since the emission granularity sets the constant.

The tight-gap floor is not the mean interarrival gap. On this corpus the mean gap is $2215\mu\text{s}$ at the median window, about $300\times$ the p_1 gap. A rule stated on the mean gap would recommend never splitting an HFT stage, and would be wrong by two orders of magnitude.

4.7 Sensitivity to h

The design equation is monotone in h : at fixed T and fixed corpus $\Delta(N)$, every κ breakpoint in the table above scales linearly with h , so halving h halves every breakpoint and moves N^* up one notch at every T for a fixed κ . At $h = 1.7\mu\text{s}$ the p_{99} -minimising N is 1, 1, 1, 2, 4, 8, 8 across the seven service floors; at a hypothetical $h = 0.5\mu\text{s}$ busy-poll mailbox it would be 1, 1, 2, 2, 4, 8, 8 — the change is a single notch, at $T = 8\mu\text{s}$, because below the tight-gap floor no reduction in h can make a split worth taking when there is no tail to remove. Reducing the hop cost widens the band of service times over which splitting pays; it does not move the threshold, which is set by the feed. For any other framework the procedure is the same: measure the one-way asynchronous hop cost h on its own dispatch path, put it into Equation (11) with the corpus Δ of Table 2, and read off N^* ; a framework whose h exceeds the tail excess a split could remove gets $N^* = 1$, and the recommendation reverses.

h is load-dependent; $1.7\mu\text{s}$ is a conservative upper bound. The $h = 1.7\mu\text{s}$ figure is the one-way cost of the shipping KASPAR-HFT BQueue mailbox measured at low load, where the receiver is sleeping on a condition variable and the send path must pay the kernel wake latency to bring it back onto CPU. That wake latency is the dominant term in the $\sim 3.4\mu\text{s}$ round-trip of Table 4 in Maciejewski [32], and the general fact that a spinning receiver delivers a message an order of magnitude faster than a wake-from-condvar receiver is standard in the microsecond-latency systems literature: David et al. [58] measure the primitive costs directly on commodity Linux hardware, Belay et al. [59] exploit this in the IX data plane by pinning busy-polling network threads, and Rigtorp’s operational guide [37] formalises the “run one thread per core and busy-wait” rule that flows from it; the fast_send paper’s own grouped-send row (Table 4, $\sim 90\text{ ns}$) is the same effect on the same framework — when the receiver never actually sleeps, the send path collapses. Under sustained load, and specifically inside a Hawkes cluster where the receiver’s queue is never empty, the receiver never sleeps: successive messages arrive faster than the receiver’s per-message service time, so it stays awake in a tight run-loop and the enqueue path reduces to a mutex acquire, a queue push, and a condition-variable notify against a receiver that is not waiting (a near-no-op). The effective h in that regime is dominated by the mutex + cache-line-transfer cost, typically a few hundred nanoseconds rather than $1.7\mu\text{s}$. This is the regime in which the tail reduction is largest, so the reported effect underestimates the benefit whenever the pipeline is inside a burst. Characterising the load-conditioned hop cost $h(\rho)$ is left to future work.

5 Evaluation: do empirics match the analytical predictions?

This section compares the empirical numbers of Section 4 against the closed-form predictions of Section 2.

Proposition 1 (burst-limit throughput). The single-message identity $p_{50}(N) = T + (N - 1)h$ is predicted exactly by Proposition 1 for any arrival law when the median message finds the queue empty. Table 1 confirms it in all 28 (T, N) cells of the Poisson arm and in 25 of the 28 Hawkes cells at $h = 1.7\mu\text{s}$, to the $0.01\mu\text{s}$ reporting precision; the three Hawkes exceptions are the longest-service cells, where the median message itself waits (Section 4.2).

Theorem 1 (reduction and scaling bound). The prediction is $\Delta(N) \leq \Delta(1)/N$ on every window, and more strongly $W_j^{(N)} \leq W_j^{(1)}/N + (N - 1)h$ for every individual message. The corpus-median ratios in Table 2 sit at or below $1/N$ in every cell, with substantial slack everywhere: the tightest ratio in the sweep is 0.163 at $T = 32$, $N = 2$, against a bound of 0.5. The slack is largest where a split drives the per-stage service under the tight-gap floor and removes the excess outright rather than dividing it, which is why the fitted γ is 3.32 and 2.68 at $T = 64$ and 128 rather than near 1. The pathwise form was checked directly by the simulator: across all 3512 windows, all seven service floors, both arms, and $N \in \{2, 4, 8\}$, no message violates the bound (the check allows N nanoseconds for the rounding of T/N ; the largest excess over that tolerance is 0). The reduction identity Equation (4) is visible in Table 2 without any fitting: Δ is constant along every diagonal of constant T/N , so the tandem’s tail is the single-server tail at s_{max} on real arrivals exactly as on synthetic ones (Appendices B and C).

Corollary 2 (Poisson null). The prediction is $p_q^{\text{P}}(N) = T + (N - 1)h$ wherever the per-stage utilisation satisfies $\rho/N < 1 - q$, and a visible wait wherever it does not. Table 1 matches both halves: Poisson p_{99} equals the floor in 22 of the 28 cells and exceeds it in exactly the six cells whose

per-stage utilisation clears 10^{-2} — $(T, N) = (32, 1), (64, 1), (64, 2), (128, 1), (128, 2)$ and $(128, 4)$, at $\rho/N = 0.0144$ to 0.0578 — and in no cell below it. Poisson $p_{99.9}$ likewise exceeds the floor exactly in the cells with $\rho/N > 10^{-3}$ (single stage at $T \geq 4$, two stages at $T \geq 8$, four stages at $T \geq 16$) and nowhere else. The null arm is therefore a working queue whose small wait appears where PASTA says it must, and splitting under it costs $(N - 1)h$ at every reported quantile.

Summary. All three analytical statements hold on the full corpus. The pathwise bound of Theorem 1 holds on every message of every window, and its slack is measured: $\gamma = 3.32$ and 2.68 at $T = 64$ and $128 \mu s$, and at $T \leq 32 \mu s$ the tandem removes the excess outright rather than dividing it.

6 Cross-validation: the same arrival process, measured live

The results of Part II so far are measurements on recorded data: pcap-derived message tapes replayed offline. That design gives scale and repeatability at the cost of external validity. A replayed tape cannot show whether the fitted clustering survives the network path to a receiver, whether it is specific to the 2025–2026 sample, or whether the service times the sweep assumes correspond to production decode times.

This section addresses all three with instrumentation of the production system. The two measurements were built for different purposes and share only the exchange they observe.

6.1 Two measurements

The live measurement instruments the KASPAR-HFT market-data path in production: a timestamp t_0 when a UDP datagram is handed up from the socket, a timestamp t_1 when the resulting book update is published, and one 16-byte record per admitted message carrying the pair together with the ring-buffer depth and the message’s position inside its datagram. It covers 8.29 million messages on six streams — ES, NQ and ZN futures, book and trade — over 53 minutes of a single afternoon. Its latency results are reported in Maciejewski [32]; we draw on them here, and on the underlying per-message records, as a control on Part II.

	Part II corpus	production measurement
source	pcap tapes, replayed offline	live multicast, production recorder
period	2025-01 to 2026-02	2026-09-16 (7 months out of sample)
instruments	NQ front month	ES, NQ, ZN $\times$ book, trade
arrival clock	CME gateway <code>sendingTime</code>	receiver t_0 at socket read
estimator	per-window, median of 3512 windows	pooled over one 53-min capture
sample	3512 windows, 276 sessions	8.29 M messages, 6 streams

Table 3. The two measurements compared in this section: different instruments, a period that does not overlap, a different arrival clock, and a different estimator.

The arrival-clock difference is the most significant. Part II keys arrivals on `sendingTime`, the timestamp the CME gateway writes into the UDP header as the packet leaves the exchange. The live measurement keys them on t_0 , taken by our own process after the packet has crossed the network, the NIC and the kernel. If the clustering reported in Section 3.2 were an artefact of exchange-side emission scheduling, the intervening path, with its own queueing, coalescing and interrupt moderation, would be expected to attenuate it. It does not.

6.2 What agrees

Table 4 puts the four statistics that both measurements compute side by side. The NQ column is the like-for-like comparison, since Part II’s corpus is NQ front month; the third column gives the range across the three live book streams.

	Part II (NQ, 3512 win)	live NQ book	live, three books
$P(\text{gap} < \text{mean}/10)$	73.2%	63.1%	63.1–84.8%
Fano(5s) collapse under shuffle	14.8×	12.2×	12.2–18.5×
Hurst exponent H	0.740	0.702	0.702–0.743
branching ratio n	0.798 (MLE)	$\leq 0.850\text{--}0.972$ (Fano ceiling)	

Table 4. The same four Poisson-rejection statistics, computed independently on recorded and on live data. The Poisson values are 9.52%, 1×, 0.5 and 0 respectively.

Three of the four are direct comparisons and agree to within the spread across live streams. The fourth compares different estimators. Part II fits the branching ratio by maximum likelihood, per window, using the Ogata recursion (Section 3.2); the live measurement does not fit a Hawkes process, and bounds n from above by inverting the Fano ceiling, $n \leq 1 - 1/\sqrt{F}$. The MLE’s 0.798 lies within every bound the live streams imply, which run from 0.850 on NQ trade to 0.972 on NQ book. The bound is loose, and the agreement is weak evidence on its own; its value is that it is independent evidence and is consistent.

The arrival process modelled in Part II is therefore not an artefact of replay, of the sample period, or of the exchange-side clock; it is present at the same strength in a live receiver seven months later on other instruments.

6.3 The service floor lands on the switch-on threshold

The live measurement supplies a quantity not obtainable from tapes: the production decode time. Isolating messages that arrive to an empty ring and first in their datagram, so that neither queueing nor in-packet serialisation contributes, gives the hot path directly. Three estimators agree:

method	ES	NQ	ZN
direct median, empty ring and first in packet	7.01	7.24	6.89
fitted intercept of the median position ladder	6.98	7.23	6.83
floor at single-message packets during an FOMC release	7.1	7.5	7.0

Table 5. Three independent estimates of the KASPAR-HFT socket-to-book floor, in μs , on the book streams. The three differ in sample, estimator and load regime. The third was taken while the packet rate rose 4.6 to 9.8× within a second; the floor moved by at most +0.4 μs and fell on two of the three instruments.

Across all six streams the fitted floor spans 6.81 to 7.23 μs , a range of 0.42 μs from six separate multicast channels sharing nothing but the binary.

Compare this with Section 4.6. The service time at which a deterministic stage on this feed begins to accumulate a tail is the feed’s tight-gap floor, its 1st-percentile interarrival gap, measured at 7.43 to 7.53 μs and invariant across intensity quintiles; the refined sweep of Section 4.5 locates the switch-on between $T = 7$ and $T = 8 \mu\text{s}$. The production decoder’s floor is 6.81–7.23 μs .

The production system operates at the threshold, to within a few hundred nanoseconds, by two measurements with no common term.

The live measurement locates the crossing. Per-packet service is the floor plus one slope for each message beyond the first (Section 6.5), so the service time of a datagram carrying span messages is floor + (span − 1) × slope. Evaluating that on each stream’s own fitted pair:

	span 1	span 2	span 3
ES book	6.98	7.55	8.11
NQ book	7.23	7.54	7.85
ZN book	6.83	7.80	8.76
ES trade	6.81	7.52	8.24
NQ trade	7.14	7.67	8.19
ZN trade	6.81	7.77	8.74
range	6.81–7.23	7.52–7.80	7.85–8.76

Table 6. Per-packet service time in μs by datagram span, from each stream’s measured floor and slope, against the feed’s tight-gap floor of 7.43–7.53 μs (Section 4.6). Every stream is below the threshold at span 1 and above it at span 2.

All six streams are below the threshold when a datagram carries one message and above it when it carries two. Six independent crossings fall in the same unit interval of span, from floors and slopes fitted separately on six multicast channels against a threshold measured on a different corpus. For this system, whether a given message is above or below the threshold is determined by whether it arrived alone in its datagram.

6.4 Where the two measurements disagree

Two quantities do not match. In both cases the discrepancy is a measurement artefact that one of the two measurements can diagnose.

The absolute Fano factor: pooling inflates it. Part II measures a corpus-median Fano at a 5-second window of 259.7; the live streams give 1024 to 1260. The ratio under the gap shuffle agrees closely (Table 4), so the discrepancy is in the level, not in the effect. The cause is the estimator row of Table 3. The live figure is computed over one uncut 53-minute capture that spans a quiet stretch and a busy one; a rate that drifts between the two inflates the count variance and reads as clustering whether or not the process is self-exciting. Part II computes the Fano factor *within* each 30-minute window and then takes the corpus median, which removes between-window rate variation by construction. Pooling raises the level by roughly a factor of four; the windowed estimate is the conservative one and is the value reported.

The shuffled Hurst null: finite-sample bias. A Fisher–Yates shuffle of a gap sequence destroys the ordering while preserving the marginal exactly, so the shuffled series is a renewal process and its Hurst exponent must be 0.5. The live measurement recovers 0.600 to 0.660 instead, and attributes the excess to finite-sample bias in the Hurst estimator at the number of windows available in a 53-minute capture and states its result as the gap between 0.74 and 0.62 rather than the distance from 0.5. Part II confirms the attribution: over 3512 windows the same shuffle returns $H = 0.501$, and the uniform-shuffle Poisson null returns $H = 0.496$ (Section 3.2). The bias is finite-sample and vanishes with more, shorter series.

6.5 Per-packet service is not constant

One assumption of the Part II simulator is contradicted by the production measurement. The sweep of Section 3.5 treats each packet as one arrival with a constant service time T . Per-packet service is not constant. A UDP datagram on this feed carries a variable number of SBE messages — call

it the packet’s *span* — which are decoded in order, so a message at position k waits for k decodes before its own. Median latency follows

$$\text{median} = \text{floor} + \text{slope} \times \text{position},$$

with the floor of Section 6.3 and a slope of 0.31 to 0.97 μs per message depending on the stream. Span is not close to constant:

stream	packets	messages	p_{50}	p_{90}	p_{99}	max
ES book	2,123,042	2,415,610	1	1	7	45
NQ book	2,506,205	3,010,036	1	2	11	36
ZN book	824,536	965,334	1	1	6	45
ES trade	83,140	235,601	3	8	17	83
NQ trade	50,279	83,233	2	7	16	85
ZN trade	22,864	100,659	2	11	71	85

Table 7. Messages per UDP datagram (span), packet-weighted quantiles. The median book packet carries one message; the 99th percentile carries 6 to 11 on the book streams and 71 on ZN trade.

Because service is charged per message, large packets account for a disproportionate share of processing time:

span ≥ 10	% of packets	% of messages	% of decode time
ES book	0.52	4.55	5.34
NQ book	1.68	4.87	2.57
ZN book	0.47	7.29	7.29
NQ trade	5.34	19.31	16.16
ES trade	6.69	26.53	40.10
ZN trade	11.31	66.18	85.54

Table 8. Concentration of work in large datagrams. On ZN trade, 11.3% of packets carry 66% of the messages and consume 86% of decode time.

The constant-service assumption is therefore violated at the packet level, and Part II’s T is the per-packet work at a typical span rather than a constant of the system. Two points follow.

First, at the message level the assumption approximately holds: the slope is fixed within a stream to the precision of the fit, and it is the composition of packets, not randomness in per-message work, that makes per-packet service vary. Theorem 1 is pathwise and arrival-law independent, and Remark 2.2 already handles unequal stages through $s_{\max}$; what the live data adds is a measured magnitude for the variability that Section 2.3 lists as an assumption.

Second, span is a separate tail mechanism on the same stream, active where the queueing mechanism is not. Section 6.6 re-runs the sweep with span-dependent service. The serialisation channel supplies the tail below the tight-gap floor, where clustering supplies none, and is negligible above it, where clustering supplies all of it; the two channels are close to independent on this corpus.

6.6 Span-dependent service

We re-extracted the per-packet span for all 3512 windows — aligned element-wise against the cached arrival arrays, with every window’s span count checked against the packet count already in the cache — and re-ran the entire sweep with service that depends on it.

Three service models are compared. All three hold the *mean* service per window at T , so the utilisation ρ is identical across them and only the distribution of work across packets differs; any difference in the tail is attributable to the span dependence.

$$\begin{array}{ll}
\text{const : } & S_i = T \quad \text{(the Part II assumption)} \\
\text{span : } & S_i = A(1 + r(\sigma_i - 1)), \quad r = 0.0432 \quad \text{(measured slope/floor)} \\
\text{prop : } & S_i = T \sigma_i / \bar{\sigma} \quad \text{(upper bound)}
\end{array}$$

with A set so that the window mean of S_i is T . The *span* model uses the ratio of slope to floor measured on NQ book in Section 6.3, 0.312/7.23. The *prop* model removes the per-packet fixed cost, making service proportional to the message count; this overstates the dependence, since the measured floor exceeds the slope by more than twenty to one, and it is included as an upper bound. As a correctness check the *const* arm reproduces the published constant-service grid of Table 1 to zero difference in all 28 cells.

Message-weighted quantiles. On this corpus 96.1% of packets carry one message and the mean span is 1.073, so a packet-weighted quantile is dominated by singletons and changes little under any service model. Weighting by message, which counts a 20-message datagram twenty times, the in-packet position distribution is concentrated at zero with a long upper tail:

message quantile	p_{50}	p_{90}	p_{95}	p_{99}	$p_{99.9}$	max
in-packet position	0	0	1	4	19	84

93.2% of messages are alone in their packet and pay no serialisation; the remaining 6.8% constitute the tail. The live measurement reports the same shape. The comparison below is message-weighted.

T	const			span			span/const		prop/const	
	p_{50}	p_{99}	$p_{99.9}$	p_{50}	p_{99}	$p_{99.9}$	p_{99}	$p_{99.9}$	p_{99}	$p_{99.9}$
2	2.00	2.00	2.00	1.99	2.34	3.54	1.17	1.77	4.64	23.35
4	4.00	4.00	4.00	3.99	4.67	7.08	1.17	1.77	4.69	26.73
8	8.00	8.56	10.10	7.97	9.63	14.87	1.12	1.47	5.21	24.64
16	16.00	34.00	53.55	15.95	35.08	56.62	1.03	1.06	3.36	11.53
32	32.00	143.74	256.51	31.89	144.41	260.14	1.00	1.01	2.37	5.88
64	98.80	815.22	1557.72	100.02	820.05	1573.97	1.01	1.01	2.37	3.94
128	366.78	5440.22	12844.51	371.91	5550.42	13159.71	1.02	1.02	2.51	3.10

Table 9. Message-weighted single-stage latency in μs , corpus median over 3512 windows, Hawkes arrivals, under constant and span-aware service at equal mean service. The last two columns give the *prop* upper bound as a ratio to const.

The two tail mechanisms occupy disjoint regimes. The span/const ratio is largest where the queueing tail is absent and smallest where the queueing tail dominates. At $T = 2$ and $4 \mu\text{s}$ the constant-service model has no tail ($p_{50} = p_{99} = p_{99.9} = T$, Section 4.1) while the span-dependent model has $p_{99} = 1.17 T$ and $p_{99.9} = 1.77 T$. Below the tight-gap floor there is a tail, but it is not a queueing tail. By $T \geq 32 \mu\text{s}$, where cluster queueing produces a p_{99} of $4.5 T$ and rising, span contributes 0–2% on top.

Decomposition at the production operating point. The live system’s measured floor is 6.81–7.23 μs (Section 6.3), so the comparable row is $T = 8 \mu\text{s}$. Decomposing the message-weighted p_{99} excess over the floor into the part the constant-service model already produced (queueing) and the remainder introduced by span-aware service (serialisation):

T	floor	queueing excess	serialisation excess	serialisation share
2	1.99	0.00	0.34	100%
4	3.99	0.00	0.69	100%
8	7.97	0.56	1.09	66%
16	15.95	18.00	1.14	6%
32	31.89	111.74	0.78	1%
64	100.02	716.42	3.61	1%
128	371.91	5073.43	105.07	2%

At $T = 8 \mu\text{s}$ serialisation supplies 66% of the p_{99} excess and queueing 34%. The live measurement reports the same ordering: evaluating the position ladder at each stream’s own p_{99} position accounts for 71% to 80% of the observed p_{99} on four of its six streams, with the remainder left to queueing and to stalls. Applying that definition to the simulator at $T = 8$ gives 97%, higher than the live figure because the only residual the model contains is queueing; it has no stall population. Both measurements attribute the larger share of the tail at the production service time to in-packet serialisation.

This is consistent with Part II. The queueing claim stands: below about 7.5 μs of service, arrival clustering does not build a queue and splitting the servicing chain removes nothing. The span-dependent run adds that a second mechanism, in-packet serialisation, is active in the same regime; it is deterministic, not a queue, and is not removed by pipelining. N^* , the p_{99} -minimising stage count, is unchanged at all seven service floors when service is made span-dependent: 1, 1, 1, 2, 4, 8, 8 under both models. The design rule of Section 10 is unaffected.

The in-packet ladder. Median latency by in-packet position at $N = 1$ reproduces the linear form of the live measurement. At $T = 8 \mu\text{s}$ the corpus-median latency by position fits

$$\text{median} = 7.98 + 0.344 \times \text{position} \quad (\mu\text{s}),$$

recovering the 0.345 μs per-message cost that was input to the model, which confirms that queueing contributes nothing at the median at this service time. The live NQ-book ladder is $7.23 + 0.312 \times \text{position}$; the forms agree and the levels differ by the difference between the 7 μs measured decode path and the 8 μs nominal service floor.

The two channels are additive, not coupled. One might suppose that span is produced by the clustering: arrivals bunch, the gateway coalesces more messages into one datagram, and a message is both queued behind a burst and positioned deep in a large packet. If so, the two mechanisms would compound and the constant-service estimates of Part II would be conservative. On this corpus this does not hold.

Two tests were made. First, a control arm in which each window’s span sequence is randomly permuted across packets, preserving the span marginal and the arrival times exactly and destroying only the association between them. If large packets arrived preferentially inside clusters, breaking that association would lower the tail. It does not: the ratio of the real to the span-shuffled p_{99} is 1.000 at every $T \leq 8 \mu\text{s}$ and lies between 0.99 and 1.02 at every larger T , with no systematic

sign. Second, measuring the association directly, the correlation between a packet's span and the logarithm of the gap that preceded it has corpus median -0.014 ; packets arriving after the tightest decile of gaps have mean span 1.010 against 1.043 for those arriving after the loosest decile, a ratio of 0.97. The association is weakly negative.

The live measurement reports the same instrument dependence: mean span rises with queue depth on ZN, where a quiet book is filled only by genuine bursts, but *falls* with queue depth on ES and NQ, whose rings back up with many small packets. Part II's corpus is NQ, and it behaves like the live NQ streams. The queueing and serialisation channels are therefore close to independent on this corpus, and the constant-service tables of Part II measure one of the two. Whether the channels couple on an instrument like ZN is open.

Part III

Best practices: implications for HFT systems design

7 The reference instance: KASPAR-HFT and its `fast_send` pipeline

The recommendations in this Part are stated for HFT systems in general and worked out on one system. KASPAR-HFT [32, 33] is an open-source C++20 trading system for CME futures built on a shared-nothing actor framework, available at <https://github.com/vincent212/kaspar-hft>: every component (socket reader, feed arbitrator, SBE decoder, order book, strategy, order manager, iLink session) is an actor with private state and a mailbox, and components interact only by passing messages. It is at once a production system (MDP3 multicast in, full order books reconstructed order by order, authenticated iLink 3 sessions out) and, with the same actor code driven from recorded packet captures, a deterministic replay simulator in its own right. Part II does not run on that simulator: its measurements are made on a standalone tandem-Lindley model of the pipeline (Section 3.3), driven by packet-arrival timestamps extracted from the same class of capture. The framework supplies the architecture under study and the calibration of h ; it is not in the measurement loop. The `fast_send` paper documents the four extensions that make the actor model usable at HFT latencies: `fast_send` itself, a synchronous delivery in which the sending thread runs the receiver's handler inline and takes the reply as a value; actor `Groups`, which co-schedule a set of actors on one thread behind a single mailbox; mailbox queue implementations selectable per actor; and a memory pool that removes per-message allocation and its tail. The defining property of `fast_send` is *receiver transparency*: a handler is written once and has no way to tell whether it was invoked synchronously or asynchronously, on which thread, or whether its sender is blocked. Section 8 builds on that property.

The reference pipeline of that paper is the design context this paper revisits. It has three sections joined by exactly two cross-thread hops. Two socket-reader actors, one per redundant multicast channel, do nothing but drain their UDP sockets and hand datagrams to a buffer actor by an asynchronous `send`; that hop is asynchronous because a reader that blocked on its callee would drop packets under a burst. From the buffer, arbitration between the A and B feeds, SBE decode, order-book construction, and signal generation run as a single chain of `fast_sends` on one thread, with no queue and no context switch between stages. When the signal decides to act it hands the order to an order-management actor by a second asynchronous `send`, because the iLink write takes microseconds and blocking the decode thread on it would stall the market-data queue. The measured

costs behind that layout are Table 4 of the **fast_send** paper: a cross-thread **send** round trip of $\sim 3.4 \mu\text{s}$ (a mailbox wakeup across cores), a same-thread grouped **send** of 90 ns, and a **fast_send** of ~ 30 ns. With a servicing chain of $\sim 7 \mu\text{s}$ measured tick-to-book on live ES, NQ, and ZN feeds, the framework’s own per-message contribution is under 1% of the floor, and the same measurement already found that the latency tail was set by the arrival process, which was strongly clustered, with a Fano-implied branching ratio of 0.85–0.97. That observation motivated the present paper.

Read against Part II, the reference pipeline is the single-thread servicing chain of Section 1.2 realised with actors: the whole T sits on one thread between two hops that were accepted because the alternative, blocking the chain, was worse. The design placed asynchronous hops where blocking was unacceptable. The present paper adds that the same logic applies inside the chain once the chain is long enough for the feed to queue behind it. The recommendations that follow are therefore stated as changes to this particular pipeline: where to cut it, in what order to place the stages, which boundaries to leave on **fast_send**, and how to decide each of those by measurement rather than by reasoning at a single call site.

The rule applied to this pipeline, as measured. The design equation can be run on the reference chain directly, using the diagonals of Table 2 as a lookup of the single-server tail excess $\Delta_{\text{single}}(s)$ against per-stage service s . Take the measured $\sim 7 \mu\text{s}$ socket-to-book chain and the natural boundaries inside it (arbitration, SBE decode, book application, signal), and suppose for the sake of the arithmetic that decode and book dominate at roughly $3 \mu\text{s}$ each with arbitration and signal near $0.5 \mu\text{s}$. Then every partition available at those cut points has $s_{\text{max}} \leq 7 \mu\text{s}$, and Part II says Δ_{single} is zero there: 0.00 at $s \leq 4 \mu\text{s}$, and at $s = 7 \mu\text{s}$ a tail on only 4.5% of windows with a median excess of 0.00 (Section 4.5). The single-thread chain has no queueing excess to remove, every cut returns exactly its hop tax, and $N^* = 1$.

This is consistent with Section 6.3: the production chain operates at the feed’s tight-gap floor, and the rule states that a stage at the floor should not be split. The live measurement adds (Section 6.6) that the tail this chain carries at its operating point is mostly in-packet serialisation, which pipelining does not reduce and per-message code optimisation (Section 10) does.

The rule applied to a longer chain. The rule applies once the chain exceeds the floor, for instance when a strategy stage is added, or on a feed with a lower gap floor. Take a $16 \mu\text{s}$ chain with the same four boundaries, decode and book now at $6 \mu\text{s}$ each and the two small stages at $2 \mu\text{s}$:

partition of the $16 \mu\text{s}$ chain	s_{max} (μs)	hops	$\Delta_{\text{single}}(s_{\text{max}})$ (μs)	p_{99} excess + hop tax
single thread (16)	16	0	18.59	18.59
8 + 8 (arb+decode book+signal)	8	1	0.61	2.31
8 + 6 + 2 (three stages)	8	2	0.61	4.01
2 + 6 + 6 + 2 (all four split)	6	3	0.00	5.10

Every entry is read directly off Table 2: $\Delta_{\text{single}}(16) = 18.59$, $\Delta_{\text{single}}(8) = 0.61$, $\Delta_{\text{single}}(6) = 0$. Three points follow. The balanced two-way cut is the best option: it removes $18 \mu\text{s}$ of p_{99} excess for $1.7 \mu\text{s}$ of hop cost, an eightfold improvement in the tail objective. Three stages do not lower s_{max} , since the $8 \mu\text{s}$ stage remains the bottleneck, so the second hop costs $1.7 \mu\text{s}$ and removes nothing (Remark 2.2, third consequence). Splitting all four lowers s_{max} to 6 and removes the remaining $0.61 \mu\text{s}$ of excess at a cost of 1.7, and is worse. The recommendation for a chain of this length is one cut, placed so that the two halves are as equal as the natural boundaries allow, with the larger half at ingress (Remark 2.2). The numbers are a prior for the end-of-build measurement of Section 8.3,

not a substitute for it. The per-stage service times are illustrative and are replaced by the operator’s profile of the chain.

8 Implementation on KASPAR-HFT: two dispatch paths, one handler

The practical claim of this paper is that on Hawkes-clustered feeds the choice of `fast_send` versus asynchronous `send` at each actor boundary is a late-stage optimisation decision, made per boundary from measured end-to-end median and tail latency of the assembled pipeline on the target feed, rather than an early architectural commitment. KASPAR-HFT supports this workflow. This section states the property that makes it possible and the procedure it enables.

The design equation maps onto framework choices in a specific way. The paper’s model of an N -stage tandem assumes each stage’s actor runs on its own thread and sends its output to the next stage through an asynchronous mailbox; the hop cost h is the wall-clock latency of that handoff.

Two hop paths in the shipping KASPAR-HFT framework. Of the delivery mechanisms described in Section 7, two matter for the design equation. The default asynchronous `send` uses `BQueue`, a blocking mailbox built on `std::mutex + std::condition_variable + boost::circular_buffer`; a producer thread enqueues under the mutex and a consumer thread waits on the condition variable. This is the hop path the paper’s Section 3.3 model corresponds to, and halving its measured $\sim 3.4\mu\text{s}$ round trip gives the one-way hop cost $h \approx 1.7\mu\text{s}$ used throughout Part II.

The `fast_send` path is different: it is a *synchronous* dispatch that acquires a per-actor mutex, calls the receiver’s handler inline on the sender’s thread, and returns the reply. There is no queue and no cross-thread wake-up. Its per-call cost is much smaller than the `BQueue` path’s, but it is not a pipelined hop: the sender blocks until the receiver finishes, so a chain of `fast_sends` serialises through all stages on one thread and offers none of the throughput speedup of Proposition 1. The paper’s design equation does not apply to `fast_send` chains.

8.1 Receiver transparency

Every actor in KASPAR-HFT registers its handlers through the `MESSAGE_HANDLER(MsgType, handler_method)` macro. A handler is a function of one message argument; nothing in its body can access, observe, or condition on the delivery mode by which it was invoked. This property is called *receiver transparency* in Maciejewski [32] and is what makes the send-mode choice a purely sender-side decision.

Concretely:

- **Synchronous delivery:** `target->fast_send(m, this)`. The sender’s thread acquires `target`’s per-actor mutex, calls the receiver’s handler inline on the sender’s thread, and takes the reply as a return value. No queue, no cross-thread wake-up. Round-trip cost $\sim 30\text{ ns}$ per Table 4 of Maciejewski [32].
- **Asynchronous delivery:** `target->send(m, this)`. The message is enqueued on `target`’s `BQueue` mailbox (`mutex + condition_variable + boost::circular_buffer`); `target`’s own thread wakes up on the condition variable and runs the handler. Round-trip cost $\sim 3.4\mu\text{s}$ across threads, or $\sim 90\text{ ns}$ same-thread within a `Group`, per the same reference.

Both paths deliver the same handler with the same message. Neither the handler’s body nor any actor downstream of it can tell which path fired. Switching a boundary between the two is a one-line change at the sender’s call site — `target->fast_send(m, this) ↔ target->send(m, this)` — with no code change on the receiver side, no header change, no interface change, no test change.

8.2 Why measurement, not local reasoning, must decide

The end-to-end latency profile of an HFT pipeline is a joint property of (i) which stages exist and where they are cut, (ii) which pairs of adjacent stages are joined by **fast_send** versus **send**, and (iii) the arrival law of the input feed. Proposition 1 and Theorem 1 both fold across multiple boundaries: the throughput speedup is N -fold and the tail contraction is $N^{-\gamma}$ with $\gamma \geq 1$, so the joint optimum is not deducible from local reasoning at a single boundary. A designer who reasons one boundary at a time underestimates the aggregate tail benefit of adding hops; this is the reasoning behind the single-thread convention of Section 1.2.

Getting the joint optimum therefore requires end-to-end measurement of the assembled pipeline against a representative feed, and iteration over per-boundary flips. The observable objective is a two-tuple (p_{50}, p_{99}) of end-to-end latency under the target feed, with an operator-chosen tradeoff between the two (Section 4.3’s κ). The design equation of Part II supplies a prior on where the optimum lies under a Hawkes feed; the measurement decides.

8.3 Operational discipline

The procedure is: draft the pipeline as a chain of actors at the natural cut points; wire each boundary with whichever of **fast_send** or **send** is convenient during development; once the pipeline runs end-to-end, measure the corpus-median p_{50} and p_{99} on a representative tape as a baseline; flip one boundary at a time, re-measure, and accept the flip if (p_{50}, p_{99}) improves under the operator’s tradeoff; iterate until no single flip improves the objective. The full design flow, including the steps that precede this, is given in Section 10.

Under the arrival law of a CME MDP3 feed the design equation of Section 4.3 predicts what the iteration converges to: **async send** at every boundary whose removal would raise the largest stage, and **fast_send** everywhere else, in particular between short sub-tasks below the floor of Section 4.3 and between sub-tasks downstream of the bottleneck whose combined service stays at or below $s_{\max}$ (Remark 2.2). The measurement decides; the prediction indicates where to expect it to converge.

8.4 Separation of code construction from latency optimization

A consequence of Section 8.3: in KASPAR-HFT two distinct topology decisions are not knowable at design time. The first is the *actor-to-thread mapping* — which actors share a thread by being placed in a common **Group**, and which run on their own thread. The second is the *per-boundary send-mode choice* — whether each pair of adjacent actors is joined by synchronous **fast_send** or asynchronous **send**. Neither decision is deducible from the source code alone. Both are functions of the target feed’s arrival statistics and the operator’s median-versus-tail tradeoff, both of which are properties of the deployment environment. The optimum can only be measured from the assembled system running against a representative feed.

Under receiver transparency this is not a limitation. Actor code, message-type headers, and the full suite of unit and integration tests can be written and run before either the actor-to-thread mapping or the send-mode choice has been decided. Code construction and testing therefore proceed independently of the latency-optimization consideration — both decisions are deferred to the end of the build, informed by measurements that could not have been taken any earlier because the system did not yet exist to measure. Correctness is established first; latency is optimized against the specific feed the system will run on, second. Two developers building two actors on the two sides of a boundary do not need to agree on whether their actors will ultimately live on the same thread or on different threads, nor on whether their handoff will be synchronous or asynchronous; they need

to agree only on the message type. The message-type agreement is checked at compile time; the two topology decisions are parameters set at deployment.

This decoupling of correctness from latency optimisation is the practical consequence of receiver transparency, and distinguishes the design cycle from that of frameworks in which either topology decision enters the receiver’s interface (Section 8.5). Design proceeds without a commitment to a topology; the commitment is made once, near deployment, on the basis of measurements the code did not need to know about while it was being written.

8.5 Contrast with non-fungible frameworks

Most low-latency actor frameworks bake the sender’s choice of delivery mode into the receiver’s type. An Akka actor’s `tell` versus `ask`, a CAF actor’s `send` versus `request`, and an Erlang process’s `cast` versus `call` each expose different receiver-side surfaces, so flipping a boundary from sync to async is a small refactor rather than a one-line change. Under those frameworks the design flow of Section 8.3 is not tractable, and the operator’s incentive is therefore to lock in the send-mode choice early — reasoning locally — rather than to defer it to an end-of-build measurement pass. This incentive is a plausible contributing cause of the single-thread convention: frameworks that penalise deferral push designers toward the topology that is least sensitive to a wrong early guess, which is the collapsed single-thread servicing chain.

KASPAR-HFT’s joint mailbox API of `fast_send` and `send`, with receiver transparency, is what makes measurement-driven send-mode optimisation practical. The design rule depends on this property operationally.

9 Other architectural considerations to re-evaluate under Hawkes

The paper’s central claim is that the single-thread hot-path recommendation does not hold above the threshold once the arrival law is Hawkes-clustered rather than Poisson (Section 1.2, Section 4). Two adjacent conventions, pre-spinning receivers and pre-warming pipelines, rest on the same Poisson-load assumption, and this section examines them. Neither intervention is measured here; both are future work. The argument is that the arrival law provides much of the benefit these interventions are intended to deliver, on the tail quantile.

Pre-spinning receivers. The busy-poll recommendation of Rigtorp [37], Belay et al. [59], and the Aeron/Chronicle lineage (Section 1.2) pins a spinning thread per receiver so the wake path from a sleeping condition variable is never taken. On a Poisson feed, any arrival can find the receiver asleep, and the wake latency is paid on average; pre-spinning eliminates it. On a Hawkes feed with branching ratio $n \approx 0.8$, sleep/wake transitions happen only at cluster boundaries. Every p_{99} message is intra-cluster (Section 4.1: the p_{99} message waits behind a cluster, since at these service times the Poisson null produces no wait at all), so by the time a tail-setting message arrives, the receiver has been continuously processing for many multiples of the per-message service time and is already effectively spinning as a natural consequence of the load. Under the model used here, pre-spinning therefore saves little or no wake latency at the tail; this is a prediction of the queueing model, not a measurement of a production receiver’s state, and should be evaluated directly. It saves wake latency for the median and for the minimum-latency floor (inter-cluster arrivals which do find the receiver asleep after seconds-scale gaps), but the saving is bounded by $\sim 1.4 \mu\text{s}$ per hop (the $1.7 \mu\text{s}$ wake-path cost less the few-hundred-nanosecond mutex and cache-line floor of Section 4.7) and is much smaller than the tail win from pipelining. There is also an operational cost: in the

present author’s production experience, pinning spinning threads across the servicing chain has caused overall system degradation — steady core burn at 100%, cache thrash between the spin loop and the actor’s own working set, and pressure on shared resources like the LLC and memory bandwidth — that offset the median-latency gains; this is anecdotal and not measured here.

Pre-warming. Pre-warming is a subset of the wake-up cost discussed above. The $h = 1.7\mu\text{s}$ one-way hop cost that this paper calibrates already captures the entire process-wake-up path on the receiver side: the kernel context switch, the reload of the receiver’s instruction cache, the L1/L2 misses on its actor state and message pool, the TLB refill, and the scheduler putting the thread back on a core. To the extent the thread hop is dominated by cluster-wait on Hawkes feeds — because every p_{99} arrival is intra-cluster and the receiver is, under the model, already awake — the cache-warmth components inside that h should follow the same pattern, since they are the same h decomposed. This is a consequence of the model rather than a measured result. Pre-warming interventions that fire synthetic traffic to keep instruction caches, data caches, TLBs, and branch predictors hot are therefore attacking the same h that Section 4 finds is dominated by cluster-wait. What such interventions can meaningfully deliver is confined to the median and the minimum — the inter-cluster first-arrival case where the receiver truly did cool down — and even there the delivery is bounded by the wake-up cost itself at the cost of continuous L1/L2 pollution from the warmup traffic. Under Hawkes the tail is unaffected.

Why this needs measurement. Treating pre-spinning and pre-warming as first-order tail-latency techniques rests, like the single-thread rule, on a Poisson-load assumption. On self-exciting exchange feeds the arrival law itself provides the receiver-awake and cache-warm state on the tail quantile, so both interventions are second-order there and can be counterproductive once their steady costs are included. Both claims are corollaries of the paper’s main result rather than separate contributions, but each deserves a dedicated empirical evaluation on Hawkes-clustered feeds, analogous to the single-thread evaluation this paper carries out. The two open questions are:

- What is the actual ROI of pinned busy-poll receivers on the Hawkes-load $p_{50}, p_{99}, \text{max}$ triple, net of the steady core-burn cost and the cache pressure the spin loop imposes on the actor’s own working set?
- What is the actual ROI of periodic pre-warming traffic on the same triple, net of the L1/L2 pollution it creates and the residual advantage it delivers only on inter-cluster first-arrivals?

Both are direct extensions of the present paper’s methodology (measure under real CME MDP3 tapes; use a matched Poisson null to isolate the arrival-law dependence) and both are candidate follow-ons.

10 Recommendations for HFT system designers

The paper’s results contradict one architectural default (single-thread hot path) and question two adjacent ones (busy-poll receivers, pre-warming). They do not contradict per-message code optimisation. This section separates what the paper leaves unchanged from what it argues against, and states the design flow the design equation implies.

10.1 What the paper leaves unchanged

Per-message hot-path optimisation — branch-free decoders, cache-line-aligned actor state, atomic-free single-writer queues, SBE decode in place, memory pools, `fast_send` at ~ 30 ns, kernel-bypass NICs (Solarflare Onload, DPDK, VMA), `isolcpus`, `SCHED_FIFO`, huge pages, RTC counters instead of `clock_gettime`, non-blocking IO on the socket, avoidance of allocation on the hot path, careful use of the branch predictor — remains necessary. All of it is contained in the service floor T , which the paper takes as given. The lower T is, the better the pipelining rule performs: Proposition 1 gives an N -factor throughput speedup at any T , and Theorem 1’s $1/N$ bound applies to a single-stage wait $\Delta(1)$ that itself scales with T . The pipelining rule is therefore complementary to per-message optimisation: a $5\mu\text{s}$ -servicing-chain implementation of a two-stage tandem beats a $10\mu\text{s}$ -servicing-chain implementation of the same two-stage tandem at every quantile, and the difference is entirely in the per-message layer.

10.2 What the paper argues against

Two conventions in the layer above per-message optimisation. First, the single-thread servicing chain as the topology default (Section 1.2): on Hawkes-clustered feeds with T above the floor of Section 4.3, the tail is dominated by cluster wait and adding threads along the chain reduces it at a median cost of $(N - 1)h$. Second, pre-spinning receivers and pre-warming pipelines as always-on interventions (Section 9): on Hawkes-clustered feeds the receiver is already busy and warm at the arrival of every p_{99} message, so these interventions affect the median and minimum, not the tail, at a steady CPU and cache cost. The claim that a faster inner loop implies a single thread is preferable conflates two claims; the paper contradicts only the second.

10.3 Recommended design flow

The design flow that follows from the paper’s results, and that we recommend as the canonical HFT infrastructure workflow on self-exciting exchange feeds:

1. **Retain per-message optimisation.** This is orthogonal to the paper’s recommendation and determines absolute latency.
2. **Measure the target feed’s tight-gap floor, and use it as the split/don’t-split test.** Take the corpus of packet arrivals, form the per-window interarrival-gap distribution, and read off a low quantile of it — we use p_1 . This single number is the service time at which a deterministic stage begins to accumulate a tail, and on CME MDP3 channel 318 it is $\approx 7.5\mu\text{s}$ (Section 4.6). A stage whose service time sits comfortably below it will not develop a queue and must not be split: the split buys nothing and costs h at every quantile. A stage above it should be split, subject to the tail excess exceeding h , which on this corpus happens about $1.5\mu\text{s}$ further along (Section 4.5). Two cautions. First, use the tight end of the gap distribution, never the mean: on this corpus the mean gap is $2215\mu\text{s}$, some $300\times$ larger, and a rule stated on it would say never to split anything. Second, this floor is a property of the exchange’s packet-emission granularity and is invariant to market intensity — it does not move between a quiet session and a volatile one (Section 4.6) — so it is measured once per feed, not tracked at runtime. Re-measure it for any feed whose gateway behaves differently.
3. **Draft the pipeline as a chain of actors at natural cut points, then balance it.** On a CME MDP3 handler: `SocketReader`, SBE decoder, order book, strategy actor, order manager — four or five stages, no artificial decomposition (Section 8). The tail factor is $s_{\max}/T$

(Remark 2.2), so the goal of the decomposition is the smallest largest stage, and the largest stage belongs at ingress so that all queueing sits in one mailbox (Remark 2.2).

4. **Wire boundaries with whichever of `fast_send` or `send` is convenient during development.** Receiver transparency in KASPAR-HFT makes this a one-line-per-boundary decision that can be deferred (Section 8).
5. **Do not commit to an actor-to-thread mapping at design time.** Grouping decisions and per-boundary send-mode decisions are both scalar dials at deployment, not code-structure decisions (Section 8.4).
6. **Measure the assembled pipeline against a representative feed.** Corpus-median p_{50} and p_{99} of end-to-end latency, per operator-chosen tradeoff (Section 8.3).
7. **Flip one boundary at a time and re-measure.** Accept the flip if (p_{50}, p_{99}) improves. Iterate until no single-boundary flip helps.
8. **Prior on where step 6 converges.** On Hawkes-clustered feeds, async `send` at every boundary whose removal would raise the largest stage; `fast_send` at every other boundary, in particular between sub-tasks downstream of the bottleneck whose combined service stays at or below $s_{\max}$, and between short sub-tasks below the floor. A hop that does not lower $s_{\max}$ costs h at every quantile and compresses nothing.
9. **Do not pre-spin or pre-warm by default.** If pre-spinning or pre-warming is used, measure the ROI on a Hawkes-representative tape, net of the steady core-burn and cache-pollution costs. If the measurement improves the tail materially, keep the intervention. The paper’s prior is that on Hawkes feeds it will not (Section 9).

The workflow prioritises measurement over local reasoning because the arrival law of the target feed is a first-order determinant of the optimum and is not a property of the code.

Part IV

Conclusions

11 Conclusion

The paper’s two conclusions contradict the prevailing view in HFT system design, which holds that the hot path should run on a single-thread event loop because every thread hop adds per-message latency. We show on CME MDP3 packet arrivals that both parts of the argument fail above a service-time threshold.

Conclusion 1: above the threshold, the hop cost is small relative to the cluster-wait tail. The dominant term in end-to-end latency on a Hawkes-clustered feed above the threshold is the cluster-wait tail, not the mailbox handoff. On 3512 windows at $h = 1.7 \mu\text{s}$ the corpus-median single-thread p_{99} is 2.2, 4.6, 12.8 and 41.5 times the service floor at $T = 16, 32, 64, 128 \mu\text{s}$, i.e. a tail excess of 11 to 2900 times the hop cost; the $p_{99.9}$ excess is roughly twice that again. Adding a hop is $+1.7 \mu\text{s}$ at every quantile; the tail it lets us compress is one to three orders of magnitude larger whenever T sits above the feed’s tight-gap floor. Below that floor the hop is cost only, so the rule is a threshold rule. The single-thread objection, that every hop is a fixed cost that no benefit recovers, does not hold on this feed above the threshold.

Conclusion 2: above the threshold, a two-stage tandem has the lower tail at the cost of one hop on the median. Splitting a service task into two half-length actors on separate threads joined by one $1.7 \mu\text{s}$ mailbox hop lowers p_{99} on 100% of the 3512 windows at every service floor from 16 to $128 \mu\text{s}$, cutting the corpus-median tail excess to between 0.03 and 0.16 of its single-thread value for a median cost of exactly h ; the trade favours the split for any tail weight $\kappa \gtrsim 0.095$ at $T = 16 \mu\text{s}$ and $\kappa \gtrsim 0.0004$ at $T = 128 \mu\text{s}$. At $T \leq 8 \mu\text{s}$, at or below the feed’s tight-gap floor, the split loses on p_{99} , as the design equation says it must. The pattern extends to $N > 2$: an N -stage tandem contracts the tail excess by a factor of N or better on every sample path (Theorem 1) — and in practice by far more than N , because a split that drives the per-stage service under the tight-gap floor removes the excess outright rather than dividing it — while paying $(N - 1)h$ at every quantile. The p_{99} -minimising stage count rises with the task length, from 1 at $T \leq 8 \mu\text{s}$ to 2 at 16, 4 at 32 and 8 at 64 and $128 \mu\text{s}$, up to the number of natural cut points in the decode chain (four or five on CME). Above the floor the single-thread event loop has the higher tail and the lower median, by one hop per stage. At the production operating point of the reference system, a socket-to-book chain of $6.8\text{--}7.2 \mu\text{s}$, $N^* = 1$ and the chain should not be split (Section 7); the recommendation applies to stages above about $10 \mu\text{s}$ on this feed.

Why the single-thread rule persists. The single-thread rule is consistent with a Poisson model of arrivals, under which there is no clustering and, at the service times a hot path runs at, the tail equals the service floor. On such a feed the rule is correct; the Poisson null gives that result for every task up to $16 \mu\text{s}$. Real exchange feeds are not Poisson: near-critical branching ratios on CME futures ($n \approx 0.9$ on ES trades in the literature, $n \approx 0.8$ on NQ packets in this corpus) have been documented for over a decade, and those clusters reverse the recommendation above the threshold. On a clustered feed the hop cost buys a shorter tail at a bounded median cost.

What this changes in practice. Multi-actor pipelines are appropriate for HFT feeds above the threshold because those feeds are self-exciting. Framework designers should measure busy-poll dequeue on pinned isolated cores against the wait/notify mailbox on a Hawkes-representative tape rather than assume it is faster (Section 9), drive the mailbox mutex uncontended, keep memory-pool discipline, and cut the decode pipeline at whichever of its natural boundaries (4–5 on CME) minimise the largest stage, with that stage at ingress and everything downstream merged up to its service time; the tail factor is $s_{\max}/T$, and a hop that does not lower $s_{\max}$ is pure cost. Strategy designers of both POV-style and race-sensitive algorithms benefit for the same reason.

Revision of a prior design context. An earlier paper by the present author [32] did not itself argue for the single-thread servicing chain; it took the collapsed single-thread servicing chain as the widely accepted design context of the time and offered `fast_send` as the mechanism that makes actor composition inside it free of context-switch overhead. What the present paper retracts is the design context, not the mechanism: on Hawkes-clustered feeds the single-thread servicing chain should no longer be the unconditional default. The corrected position is that `fast_send` (synchronous, inline, sub-microsecond) and asynchronous `BQueue` handoffs (a real pipelined hop at a few microseconds) are *complementary* tools, and the choice between them is a per-boundary design decision that depends on the service time being chained and the arrival law of the feed the pipeline sits on.

In shipping KASPAR-HFT the `fast_send` vs asynchronous-send choice at each actor boundary is normally left open until the end of the build, and is resolved by measuring the end-to-end median and tail latency of the assembled pipeline under the target feed. A convenient property of the KASPAR-HFT actor API is that the receiving actor’s `MESSAGE_HANDLER` is agnostic to how it was invoked: swapping a synchronous `fast_send` at a call site for an asynchronous `send` (or vice versa) is a one-line change on the sender side and requires no code change on the receiver side. The choice therefore lives entirely at each individual call site and can be flipped per boundary at end-of-build with minimal engineering cost. The paper’s design equation now supplies the arrival-law-aware default value of that end-of-build measurement: on Hawkes-clustered feeds, long tasks should be split into shorter actors joined by asynchronous sends as a matter of preference, on the presumption that splitting compresses the overall tail more than the added hop cost inflates the median. The rule presupposes that the task being split has a service time T comfortably above the floor of Section 4.3; a task of order h or below cannot be usefully split (a $1\ \mu\text{s}$ task cannot be turned into two $500\ \text{ns}$ actors when a single hop alone costs $1.7\ \mu\text{s}$). Short synchronous chains inside one actor’s servicing of one message therefore continue to use `fast_send` unchanged. The end-of-build measurement decides any specific boundary.

Main caveats. The corpus is a single instrument (NQ front-month, chan 318); ES and BTC replications are follow-on work. Constant per-packet service is a simplification; Section 6.6 removes it for the one source of variability the live data lets us measure, the message count per datagram, and finds the design rule unchanged, but per-message work that varies with book state is still not modelled. The cache-pollution cost of spinning receivers on many cores is bounded but not measured. And Theorem 1 is a bound whose slack depends on the arrival law; the paper measures that slack ($\gamma = 3.3$ and 2.7 where it is defined, against the guaranteed $\gamma \geq 1$) rather than deriving it, and a closed form for the slack on Hawkes input is open.

None of these caveats reverses the two conclusions on this corpus.

12 Future work

- Multi-instrument replication (ES, BTC).
- Closed form for the slack in Theorem 1 under Hawkes input, i.e. the exponent $\gamma - 1$ as a function of $(\rho, n, \beta T)$. By the reduction this is a single-server question about $M^{[K]}/D/1$ with Borel batches at two service levels, which is tractable; the paper measures γ instead.
- Span/clustering coupling on a coalescing-heavy feed. The span-aware rerun of Section 6.6 finds the serialisation and queueing channels close to independent on NQ, matching the live NQ streams; the live ZN streams show span rising steeply with queue depth instead. Whether the two channels compound on an instrument of that kind, and what that does to the design rule, is not answered here.
- Cache-pollution empirical measurement per spinning-receiver stage count.
- Power-law-kernel Hawkes fit versus exponential.
- Overnight-session regime.
- Own-capture pcap validation of the network sub-stage.
- Load-conditioned hop cost $h(\rho)$. This paper calibrates $h = 1.7 \mu\text{s}$ from the low-load wake-latency figure in Maciejewski [32]; under sustained-load conditions inside a Hawkes cluster the receiver never sleeps and the effective h collapses toward the mutex + cache-line-transfer floor of a few hundred nanoseconds (Section 4.7). Measuring $h(\rho)$ directly and re-running the sweep against the load-conditioned h would sharpen the reported tail-win, which the current single- h sim understates.

13 Reproducibility

All results in Part II are reproducible from the KASPAR-HFT repository [33], <https://github.com/vincent212/kaspar-hft>, directory `arrival_paper/`. `qsim_prep.py` extracts per-window packet arrivals keyed on `sendingTime`, fits the packet-Hawkes model and writes the cache; `qsim_run.py` runs the (T, h, N) tandem Lindley grid with the Poisson null and the per-window bound check; `make_spans.py`, `qsim_span.py` and `make_span_tables.py` produce the span-aware rerun of Section 6.6; `make_tables.py`, `make_figure.py` and `make_diagnostics.py` regenerate the tables, Figure 2 and the Section 3.2 diagnostics from the cache. The Poisson null is seeded per window from an MD5 of `(session, window_id)`, so the cache is byte-reproducible.

The live measurement of Section 6 uses the `LatencyProbe` actor in the same repository and the readers under `kaspr/perf/`; its latency results are published in Maciejewski [32].

The 281 session tapes are derived from licensed CME MDP3 channel 318 pcaps (NQ front month, 2025-01 through 2026-02) and are not redistributed; the extraction script and the cached per-window metadata are released.

References

- [1] Hawkes, A. G. (1971). Spectra of some self-exciting and mutually exciting point processes. *Biometrika* 58, 83–90.

- [2] Hawkes, A. G., Oakes, D. (1974). A cluster process representation of a self-exciting process. *Journal of Applied Probability* 11(3), 493–503.
- [3] Tanner, J. C. (1961). A derivation of the Borel distribution. *Biometrika* 48(1–2), 222–224.
- [4] Asmussen, S. (2003). *Applied Probability and Queues*, 2nd ed. Springer. Chapter X (heavy-tailed and subexponential asymptotics for single-server queues).
- [5] Gao, X., Zhu, L. (2018). Functional central limit theorems for stationary Hawkes processes and application to infinite-server queues. *Queueing Systems* 90, 161–206.
- [6] Jackson, J. R. (1957). Networks of waiting lines. *Operations Research* 5, 518–521.
- [7] Burke, P. J. (1956). The output of a queuing system. *Operations Research* 4, 699–704.
- [8] Kelly, F. P. (1979). *Reversibility and Stochastic Networks*. Wiley.
- [9] Kleinrock, L. (1976). *Queueing Systems, Vol. II: Computer Applications*. Wiley.
- [10] Whitt, W. (1983). The queueing network analyzer. *Bell System Technical Journal* 62, 2779–2815.
- [11] Friedman, H. D. (1965). Reduction methods for tandem queueing systems. *Operations Research* 13(1), 121–131.
- [12] Lindley, D. V. (1952). The theory of queues with a single server. *Mathematical Proceedings of the Cambridge Philosophical Society* 48(2), 277–289.
- [13] Foss, S., Korshunov, D. (2000). Steady-state asymptotics for tandem, split-match and other feedforward queues with heavy-tailed service. *Queueing Systems* 35, 65–80.
- [14] Baccelli, F., Foss, S., Lelarge, M. (2005). Tail asymptotics for monotone-separable networks. *J. Appl. Probability* 42(3), 793–812.
- [15] Heindl, A. (2001). Decomposition of general tandem queueing networks with MMPP input. *Performance Evaluation* 44(1–4), 5–23.
- [16] Kim, Y. H., Chydzinski, A. (2002). Connection-wise end-to-end performance analysis of queueing networks with MMPP inputs. *Performance Evaluation* 47(2–3), 137–156.
- [17] Dębicki, K., Mandjes, M. (2015). *Queues and Lévy Fluctuation Theory*. Springer.
- [18] Daw, A., Pender, J. (2018). Queues driven by Hawkes processes. *Stochastic Systems* 8(3), 192–229.
- [19] Koops, D. T., Boxma, O. J., Mandjes, M. R. H. (2018). Infinite-server queues with Hawkes arrivals. *Queueing Systems* 92, 27–52.
- [20] Chen, X., Blanchet, J. (2021). Perfect sampling of Hawkes processes and queues with Hawkes arrivals. *Stochastic Systems* 11(3), 264–283.
- [21] Sheldon, D., et al. (2023). Steady-state analysis and online learning for queues with Hawkes arrivals. arXiv 2311.02577.
- [22] Gao, X., Zhu, L. (2024). Single-server queues with state-dependent Hawkes arrivals. *Mathematics of Operations Research*.

- [23] Ozaki, T. (1979). Maximum likelihood estimation of Hawkes’ self-exciting point processes. *Ann. Inst. Statist. Math.* 31(1), 145–155.
- [24] Filimonov, V., Sornette, D. (2012). Quantifying reflexivity in financial markets: toward a prediction of flash crashes. *Physical Review E* 85, 056108.
- [25] Hardiman, S. J., Bercot, N., Bouchaud, J.-P. (2013). Critical reflexivity in financial markets: a Hawkes process analysis. arXiv 1302.1405.
- [26] Bacry, E., Mastromatteo, I., Muzy, J.-F. (2015). Hawkes processes in finance. *Market Microstructure and Liquidity* 1(1).
- [27] Byrd, D., Hybinette, M., Balch, T. H. (2020). ABIDES: Towards high-fidelity market simulation for AI research.
- [28] Frey, S., Li, K., Nagy, P., Sapora, S., Lu, C., Zohren, S., Foerster, J., Calinescu, A. (2023). JAX-LOB. arXiv 2308.13289.
- [29] Aquilina, M., Budish, E., O’Neill, P. (2022). Quantifying the high-frequency trading “arms race”. *Quarterly Journal of Economics* 137(1), 493–564.
- [30] Dean, J., Barroso, L. A. (2013). The tail at scale. *Communications of the ACM* 56(2), 74–80.
- [31] Zhao, K., Alizadeh, M., Pan, R., Ports, D. R. K., Perry, J., Yang, X., et al. (2023). Parsimon: scalable tail latency estimation for data center networks. NSDI 2023.
- [32] Maciejewski, V. (2026). Adapting the actor model of concurrency for high-frequency trading: synchronous message delivery (`fast_send`) and a tick-to-book latency study. arXiv:2609.21173. <https://arxiv.org/abs/2609.21173>
- [33] Maciejewski, V. (2026). KASPAR-HFT: an open-source C++20 actor framework and trading system for CME futures. Source repository. <https://github.com/vincent212/kaspar-hft>
- [34] Thompson, M. (2011). The Single Writer Principle. *Mechanical Sympathy*, 29 September 2011. <https://mechanical-sympathy.blogspot.com/2011/09/single-writer-principle.html>
- [35] Fowler, M. (2011). The LMAX Architecture. 12 July 2011. <https://martinfowler.com/articles/lmax.html>
- [36] LMAX Exchange (2011). Disruptor: high-performance alternative to bounded queues for exchanging data between concurrent threads. <https://lmax-exchange.github.io/disruptor/>
- [37] Rigtorp, E. (2020). Low latency tuning guide. <https://rigtorp.se/low-latency-guide/>
- [38] Chronicle Software (2023). Building fast trading engines: Chronicle’s approach to low-latency trading. <https://chronicle.software/insights/blogs/building-fast-trading-engines-chronicles-approach-to-low-latency-trading>
- [39] Real Logic (2016). Aeron: efficient reliable UDP unicast, UDP multicast, and IPC message transport. <https://aeron.io/>
- [40] Kwan, A., Wang, H., Luk, W. (2023). C++ design patterns for low-latency applications including high-frequency trading. arXiv 2309.04259.

- [41] Hu, W. (2024). The deterministic event-driven sequencer architecture: a competitive edge for high-frequency trading. Medium, October 2024. <https://medium.com/@hu.wenzhe124124/the-deterministic-event-driven-sequencer-architecture-a-competitive-edge-for-high-frequency>
- [42] Schmidt, D. C. (1995). Reactor: an object behavioral pattern for concurrent event demultiplexing and event handler dispatching. In *Pattern Languages of Program Design*, J. O. Coplien and D. C. Schmidt (eds.), Addison-Wesley.
- [43] Kegel, D. (1999). The C10K problem. <https://www.kegel.com/c10k.html>
- [44] Sysoev, I. (2004). nginx: a high-performance HTTP server and reverse proxy. <https://nginx.org/>
- [45] Dahl, R. (2009). Node.js: JavaScript runtime built on the V8 engine. Presented at JSConf EU, Berlin, November 2009. <https://nodejs.org/>
- [46] Welsh, M., Culler, D., Brewer, E. (2001). SEDA: an architecture for well-conditioned, scalable internet services. *Proc. 18th ACM Symposium on Operating Systems Principles (SOSP)*, 230–243.
- [47] Welsh, M. (2010). A retrospective on SEDA. <https://matt-welsh.blogspot.com/2010/07/retrospective-on-seda.html>
- [48] Kohler, E., Morris, R., Chen, B., Jannotti, J., Kaashoek, M. F. (2000). The Click modular router. *ACM Transactions on Computer Systems* 18(3), 263–297.
- [49] Murray, D. G., McSherry, F., Isaacs, R., Isard, M., Barham, P., Abadi, M. (2013). Naiad: a timely dataflow system. *Proc. 24th ACM Symposium on Operating Systems Principles (SOSP)*, 439–455.
- [50] Marty, M., de Kruijf, M., Adriaens, J., Alfeld, C., et al. (2019). Snap: a microkernel approach to host networking. *Proc. 27th ACM Symposium on Operating Systems Principles (SOSP)*, 399–413.
- [51] Kaffes, K., Chong, T., Humphries, J. T., Belay, A., Mazières, D., Kozyrakis, C. (2019). Shinjuku: preemptive scheduling for μ second-scale tail latency. *Proc. 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 345–359.
- [52] Ousterhout, A., Fried, J., Behrens, J., Belay, A., Balakrishnan, H. (2019). Shenango: achieving high CPU efficiency for latency-sensitive datacenter workloads. *Proc. 16th USENIX NSDI*, 361–378.
- [53] Fried, J., Ruan, Z., Ousterhout, A., Belay, A. (2020). Caladan: mitigating interference at microsecond timescales. *Proc. 14th USENIX OSDI*, 281–297.
- [54] Chen, H., Delimitrou, C., Kozyrakis, C. (2021). Perséphone: a microsecond-scale time-sharing scheduler for wide service-time distributions. *Proc. 28th ACM SOSP*, 653–668.
- [55] DPDK Project (2024). DPDK Programmer’s Guide: run-to-completion and pipeline modes. https://doc.dpdk.org/guides/prog_guide/
- [56] Zhang, X. (2016). Pipelined FAST protocol market-data decoder for high-frequency trading. Chinese patent CN105654383A.

- [57] Vinnikov, A. (2014). Batch latency analysis and phase transitions for a tandem of queues. arXiv 1403.2400.
- [58] David, T., Guerraoui, R., Trigonakis, V. (2013). Everything you always wanted to know about synchronization but were afraid to ask. *Proc. 24th ACM Symposium on Operating Systems Principles (SOSP)*, 33–48.
- [59] Belay, A., Prekas, G., Klimovic, A., Grossman, S., Kozyrakis, C., Bugnion, E. (2014). IX: a protected dataplane operating system for high throughput and low latency. *Proc. 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 49–65.

Appendices

A The Borel batch-size tail at fixed branching ratio

This appendix records the calculation behind Equation (8) and the numerical check that the critical $k^{-1/2}$ survival tail of the Borel distribution is confined to a finite window at every $n < 1$.

Stirling's formula $k! = \sqrt{2\pi k} k^k e^{-k} (1 + O(1/k))$ in Equation (7) gives

$$\Pr(K = k) = \frac{(nk)^{k-1} e^{-nk}}{k!} = \frac{n^{k-1} k^{k-1} e^{-nk}}{\sqrt{2\pi k} k^k e^{-k}} (1 + O(1/k)) = \frac{1}{n\sqrt{2\pi}} k^{-3/2} (ne^{1-n})^k (1 + O(1/k)).$$

Writing $n = 1 - \epsilon$, $\log(ne^{1-n}) = \log(1 - \epsilon) + \epsilon = -\epsilon^2/2 - \epsilon^3/3 - \dots$, so the geometric factor is $\exp(-k\epsilon^2/2 (1 + O(\epsilon)))$. The pmf is therefore $\propto k^{-3/2}$ for $k \ll k^* = \epsilon^{-2}$ and geometric beyond; summing, $\Pr(K > k) \asymp k^{-1/2}$ on the same window and $\asymp k^{-3/2} e^{-k\epsilon^2/2}$ beyond it. At $n = 1$ exactly, $k^* = \infty$, $\Pr(K > k) \sim \sqrt{2/\pi} k^{-1/2}$ (Borel–Tanner), and $E[K] = \infty$; that is the unstable case $\rho = \infty$, which no stationary queue attains.

Because the tail is geometric, K has finite exponential moments $E[e^{\theta K}] < \infty$ for $\theta < \epsilon^2/2 (1 + O(\epsilon))$ and is therefore outside the subexponential class $\mathcal{S}$. Heavy-tail queueing results that assume subexponential batch work, including the single-big-batch principle for $M^{[K]}/G/1$ and the monotone-separable tail theorem of Baccelli et al. [14], do not apply, and the stationary waiting time of the batch-arrival queue has a Cramér–Lundberg (exponential) tail at every fixed $n < 1$.

Table 10 reports the local log-log slope of the exact pmf, $\Delta \log \Pr(K = k) / \Delta \log k$ between successive k , which a pure $k^{-3/2}$ law would hold at -1.5 .

n	k^*	$10 \rightarrow 30$	$30 \rightarrow 100$	$100 \rightarrow 300$	$300 \rightarrow 1000$	$1000 \rightarrow 3000$
0.80	25	−1.92	−2.84	−5.71	−14.96	−43.63
0.90	100	−1.59	−1.81	−2.48	−4.62	−11.26
0.95	400	−1.52	−1.57	−1.73	−2.25	−3.85
0.99	10000	−1.50	−1.50	−1.51	−1.53	−1.59

Table 10. Local log-log slope of the Borel pmf, Equation (7), between successive cluster sizes k . The critical slope -1.5 holds while $k \ll k^* = (1 - n)^{-2}$ and steepens geometrically beyond.

The practical reading is that at the branching ratios fitted on the corpus ($n_{\text{pkt}} \approx 0.8$, Section 3.2), $k^* \approx 25$: clusters of order a hundred packets are the largest that occur with non-negligible frequency, and the single-stage wait behind the largest clusters has scale $Tk^* \approx 25T$, against the corpus-median p_{99} tail excess of Section 4, which is zero below the tight-gap floor and rises from $1.2T$ at $T = 16 \mu\text{s}$ to $39T$ at $T = 128 \mu\text{s}$. This is the object that Theorem 1 divides by N , and the reason the $n = 0.80$ row of Table 10 is the relevant one.

B Numerical check of Theorem 1

A direct simulation of the model of Theorem 1 checks both the identity Equation (4) and the bound Equation (6). Immigrants arrive as a Poisson process at rate $\mu = 0.004 \mu\text{s}^{-1}$, each spawning a Galton–Watson cluster with Poisson(0.9) offspring, so $K \sim \text{Borel}(0.9)$ and $E[K] \approx 10$; clusters are placed as instantaneous batches, giving $\rho = \bar{\lambda}T \approx 0.40$ at $T = 10 \mu\text{s}$. The tandem has $N = 4$ equal stages of $2.5 \mu\text{s}$ and $h = 1.7 \mu\text{s}$; 2×10^5 messages are simulated with an explicit per-stage FIFO recursion

(independent of the paper’s `qsim_run.py`). Over all messages, $\max_j |W_j^{(4)} - (w_j(T/4) + 3h)| < 10^{-9} \mu s$ and Equation (6) is violated by no message. The quantiles are

q	$W_q^{(1)}$	$W_q^{(4)}$	$W_q^{(1)}/4 + 3h$	$(W_q^{(4)} - 3h)/W_q^{(1)}$
0.5	350.0	55.1	92.6	0.14
0.9	2066.6	355.1	521.8	0.17
0.99	5342.1	1127.6	1340.6	0.21
0.999	7586.2	1585.1	1901.7	0.21

in microseconds. The tandem lands below the bound at every quantile, with the ratio in the last column between 0.14 and 0.21 against the bound’s 0.25; the slack is the ρ/N utilisation effect described in the remark after Theorem 1. Under the Poisson null at the same message rate ($\rho = 0.40$, which is far above the corpus utilisation and is used here only to make the Poisson wait visible) the single-stage wait quantiles at $q = 0.9, 0.99, 0.999$ are $10.3, 24.8, 38.5 \mu s$ and the tandem’s are $5.1, 7.5, 9.2 \mu s$, again inside the bound. At the corpus utilisation ($\rho = 0.0045$ at this appendix’s $T = 10 \mu s$) the same two collapse to 0 and $3h$ at $q = 0.9$ and $q = 0.99$, which is Corollary 2; at $q = 0.999$ they do not, since ρ there exceeds $1 - q = 0.001$ and the corollary does not apply.

C Numerical check of the unequal-stage claims

The same generator as Appendix B (Borel(0.9) clusters, $\rho = 0.40$, $T = 10 \mu s$, $h = 1.7 \mu s$), with each cluster’s members spread uniformly over $3 \mu s$ instead of placed as an instantaneous batch and 4×10^5 messages, run through the partitions of Remark 2.2 with an explicit per-stage FIFO recursion. The last two columns are the tail-excess ratio $(\Delta_{99}(P) - \text{hops} \cdot h)/\Delta_{99}(\text{single})$ and the bound $s_{\max}/T$ from Theorem 1.

partition	$s_{\max}$	hops	p_{50}	p_{99}	$p_{99.9}$	ratio	bound
10	10.0	0	388.2	5719.0	8249.3	1.00	1.00
7 + 3	7.0	1	213.6	3429.5	5297.1	0.60	0.70
3 + 7	7.0	1	213.6	3429.5	5297.1	0.60	0.70
5 + 5	5.0	1	134.9	2265.5	3574.2	0.39	0.50
7 + 2 + 1	7.0	2	215.3	3431.2	5298.8	0.60	0.70
4 + 3 + 3	4.0	2	104.6	1746.5	2714.6	0.30	0.40
4 + 3 + 2 + 1	4.0	3	106.3	1748.2	2716.3	0.30	0.40
2.5×4	2.5	3	66.4	1049.8	1562.2	0.18	0.25

Latencies are end-to-end in microseconds. The three claims of Remark 2.2 read off directly: the tail excess depends on the partition only through $s_{\max}$ and sits below the bound at every row; $7 + 3$ and $3 + 7$ are identical to the reported precision; and $7 + 2 + 1$ differs from $7 + 3$, and $4 + 3 + 2 + 1$ from $4 + 3 + 3$, by exactly one hop cost $h = 1.7 \mu s$ at every quantile. The $3 \mu s$ intra-cluster spread, which is above $s_{\max}$ for the last row, is why the equal-stage ratio here (0.18) sits slightly below the batch-input value of Appendix B (0.21): a cluster that arrives spread out lets the faster tandem drain part of it before the next member lands.